College DBMS Project Source Documentation

Blood Donation Management System

Complete Frontend Codebase (React 18 + Vite + TypeScript + Tailwind CSS)

Index of Frontend Source Files

frontend/package.json
frontend/vite.config.ts
frontend/tsconfig.json
frontend/tailwind.config.js
frontend/postcss.config.js
frontend/index.html
frontend/src/vite-env.d.ts
frontend/src/index.css
frontend/src/main.tsx
frontend/src/App.tsx
frontend/src/types/index.ts
frontend/src/api/client.ts
frontend/src/context/AuthContext.tsx
frontend/src/utils/errorHandler.ts
frontend/src/layouts/MainLayout.tsx
frontend/src/pages/Login.tsx
frontend/src/pages/Dashboard.tsx
frontend/src/pages/Donors.tsx
frontend/src/pages/Donations.tsx
frontend/src/pages/Inventory.tsx
frontend/src/pages/Requests.tsx
frontend/src/pages/Events.tsx
frontend/src/pages/Hospitals.tsx
frontend/src/pages/Reports.tsx
frontend/src/pages/Users.tsx
frontend/src/pages/Settings.tsx

frontend/package.json 32 lines

```
{
  "name": "blood-donation-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.6.8",
    "clsx": "^2.1.0",
    "lucide-react": "^1.61.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.22.3",
    "recharts": "^2.12.3",
    "tailwind-merge": "^2.2.2"
  },
  "devDependencies": {
    "@types/react": "^18.2.66",
    "@types/react-dom": "^18.2.22",
    "@vitejs/plugin-react": "^4.2.1",
    "autoprefixer": "^10.4.19",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.1",
    "typescript": "^5.2.2",
    "vite": "^5.1.6"
  }
}
```

frontend/vite.config.ts 11 lines

```ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    host: true
  }
})
```

frontend/tsconfig.json 20 lines

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src"]
}
```

frontend/tailwind.config.js 21 lines

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {
      colors: {
        blood: {
          50: '#fef2f2',
          100: '#fee2e2',
          500: '#ef4444',
          700: '#B91C2C', // Primary red
          900: '#7F1D1D', // Burgundy
        }
      }
    },
  },
  plugins: [],
}
```

frontend/postcss.config.js 6 lines

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

frontend/index.html 13 lines

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="data:image/svg+xml,<svg xmlns=%22http://www.w3.org/2000/svg%22 viewBox=%220%22%20viewBox=%220%200%20100%20100%22><text y=%22.9em%22 font-size=%2290%22>🩸</text></svg>" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Blood Donation Management System</title>
  </head>
  <body class="bg-slate-50 text-slate-900 font-sans antialiased">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

frontend/src/vite-env.d.ts 1 lines

```
1  /// <reference types="vite/client" />
```

frontend/src/index.css 9 lines

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  margin: 0;
  min-height: 100vh;
  background-color: #F8FAFC;
}
```

frontend/src/main.tsx 10 lines

```tsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

frontend/src/App.tsx 59 lines

```tsx
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './context/AuthContext';
import { MainLayout } from './layouts/MainLayout';
import { Login } from './pages/Login';
import { Dashboard } from './pages/Dashboard';
import { Donors } from './pages/Donors';
import { Donations } from './pages/Donations';
import { Inventory } from './pages/Inventory';
import { Requests } from './pages/Requests';
import { Events } from './pages/Events';
import { Hospitals } from './pages/Hospitals';
import { Reports } from './pages/Reports';
import { UsersPage } from './pages/Users';
import { SettingsPage } from './pages/Settings';

const ProtectedRoute: React.FC<{ children: React.ReactNode; adminOnly?: boolean }> = ({
  children,
  adminOnly = false
}) => {
  const { isAuthenticated, isAdmin } = useAuth();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  if (adminOnly && !isAdmin) {
    return <Navigate to="/" replace />;
  }

  return <MainLayout>{children}</MainLayout>;
};

export const App: React.FC = () => {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<Login />} />

          <Route path="/" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
          <Route path="/donors" element={<ProtectedRoute><Donors /></ProtectedRoute>} />
          <Route path="/donations" element={<ProtectedRoute><Donations /></ProtectedRoute>} />
          <Route path="/inventory" element={<ProtectedRoute><Inventory /></ProtectedRoute>} />
          <Route path="/requests" element={<ProtectedRoute><Requests /></ProtectedRoute>} />
          <Route path="/events" element={<ProtectedRoute><Events /></ProtectedRoute>} />
          <Route path="/hospitals" element={<ProtectedRoute><Hospitals /></ProtectedRoute>} />
          <Route path="/reports" element={<ProtectedRoute><Reports /></ProtectedRoute>} />
          <Route path="/users" element={<ProtectedRoute adminOnly><UsersPage /></ProtectedRoute>} />
          <Route path="/settings" element={<ProtectedRoute><SettingsPage /></ProtectedRoute>} />

          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
};
```

```
58
59  export default App;
```

frontend/src/types/index.ts 158 lines

```ts
export interface User {
  id: number;
  full_name: string;
  email: string;
  role: 'admin' | 'staff';
  is_active: boolean;
  created_at: string;
}

export interface AuthResponse {
  access_token: string;
  token_type: string;
  role: 'admin' | 'staff';
  full_name: string;
  email: string;
}

export interface Donor {
  id: number;
  donor_code: string;
  full_name: string;
  email?: string;
  phone: string;
  date_of_birth: string;
  gender: 'Male' | 'Female' | 'Other';
  blood_group: string;
  address?: string;
  city: string;
  is_eligible: boolean;
  last_donation_date?: string;
  notes?: string;
  is_active: boolean;
  created_at: string;
}

export interface Hospital {
  id: number;
  hospital_code: string;
  name: string;
  license_number: string;
  contact_person: string;
  phone: string;
  email: string;
  address: string;
  city: string;
  is_active: boolean;
  created_at: string;
}

export interface DonationEvent {
  id: number;
  event_code: string;
  title: string;
  organizer: string;
  venue: string;
  city: string;
  event_date: string;
  start_time: string;
  end_time: string;
```

```
  capacity: number;
  status: 'Upcoming' | 'Ongoing' | 'Completed' | 'Cancelled';
  description?: string;
  registered_count: number;
  created_at: string;
}

export interface EventRegistration {
  id: number;
  event_id: number;
  donor_id: number;
  registered_at: string;
  status: 'Registered' | 'Attended' | 'Cancelled';
  donor_name?: string;
  donor_phone?: string;
  blood_group?: string;
}

export interface Donation {
  id: number;
  donation_code: string;
  donor_id: number;
  donor_name?: string;
  event_id?: number;
  donation_date: string;
  blood_group: string;
  component_type: string;
  units: number;
  hemoglobin_g_dl?: number;
  blood_pressure?: string;
  pulse_rate?: number;
  screening_status: 'Passed' | 'Failed' | 'Deferred';
  status: 'Scheduled' | 'Completed' | 'Rejected' | 'Cancelled';
  inventory_generated: boolean;
  remarks?: string;
  recorded_by_name?: string;
  created_at: string;
}

export interface BloodInventoryItem {
  id: number;
  batch_number: string;
  donation_id?: number;
  blood_group: string;
  component_type: string;
  volume_ml: number;
  collection_date: string;
  expiry_date: string;
  status: 'Available' | 'Reserved' | 'Issued' | 'Expired' | 'Discarded';
  storage_location: string;
  created_at: string;
}

export interface BloodRequest {
  id: number;
  request_code: string;
  hospital_id: number;
  hospital_name?: string;
  patient_name?: string;
  blood_group: string;
  component_type: string;
  units_requested: number;
  units_fulfilled: number;
```

```
  urgency: 'Normal' | 'Urgent' | 'Critical';
  required_date: string;
  status: 'Pending' | 'Approved' | 'Partially Fulfilled' | 'Fulfilled' | 'Rejected' | 'Cancelled';
  notes?: string;
  created_at: string;
}

export interface BloodIssue {
  id: number;
  issue_code: string;
  request_id: number;
  units_issued: number;
  issued_date: string;
  issued_by_name?: string;
  recipient_name: string;
  notes?: string;
}

export interface DashboardStats {
  total_donors: number;
  eligible_donors: number;
  available_units: number;
  pending_requests: number;
  donations_this_month: number;
  inventory_by_group: { blood_group: string; units: number }[];
  monthly_trends: { month: string; donations: number }[];
  recent_donations: any[];
  latest_requests: any[];
}

export interface SystemSetting {
  setting_key: string;
  setting_value: string;
  description?: string;
  updated_at: string;
}
```

frontend/src/api/client.ts 36 lines

```ts
import axios from 'axios';

const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://127.0.0.1:8000/api';

export const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor to attach JWT token
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response interceptor to handle unauthorized redirects gracefully
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      if (window.location.pathname !== '/login') {
        window.location.href = '/login';
      }
    }
    return Promise.reject(error);
  }
);

export default apiClient;
```

frontend/src/context/AuthContext.tsx 79 lines

```tsx
import React, { createContext, useContext, useState, useEffect } from 'react';
import { User, AuthResponse } from '../types';
import apiClient from '../api/client';

interface AuthContextType {
  user: User | null;
  token: string | null;
  login: (authData: AuthResponse) => void;
  logout: () => void;
  isAuthenticated: boolean;
  isAdmin: boolean;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [user, setUser] = useState<User | null>(() => {
    const saved = localStorage.getItem('user');
    return saved ? JSON.parse(saved) : null;
  });
  const [token, setToken] = useState<string | null>(() => localStorage.getItem('token'));

  useEffect(() => {
    if (token && !user) {
      apiClient.get('/auth/me')
        .then((res) => {
          setUser(res.data);
          localStorage.setItem('user', JSON.stringify(res.data));
        })
        .catch(() => logout());
    }
  }, [token]);

  const login = (authData: AuthResponse) => {
    setToken(authData.access_token);
    const userInfo: User = {
      id: 0,
      full_name: authData.full_name,
      email: authData.email,
      role: authData.role,
      is_active: true,
      created_at: new Date().toISOString()
    };
    setUser(userInfo);
    localStorage.setItem('token', authData.access_token);
    localStorage.setItem('user', JSON.stringify(userInfo));
  };

  const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    window.location.href = '/login';
  };

  return (
    <AuthContext.Provider
      value={{
```

```
        user,
        token,
        login,
        logout,
        isAuthenticated: !!token,
        isAdmin: user?.role === 'admin'
      }}
    >
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};
```

frontend/src/utils/errorHandler.ts 39 lines

```ts
/**
 * Extracts a readable error string from any API response, preventing
 * React crashes from rendering raw error objects or Pydantic validation lists.
 */
export function formatApiError(error: any, fallback: string = 'An unexpected error occurred.'): string
{
  if (!error) return fallback;

  const detail = error.response?.data?.detail;

  if (typeof detail === 'string') {
    return detail;
  }

  // Handle FastAPI / Pydantic validation errors: array of { loc, msg, type, ... }
  if (Array.isArray(detail)) {
    return detail
      .map((item: any) => {
        if (typeof item === 'string') return item;
        const field = Array.isArray(item.loc) ? item.loc.slice(1).join(' -> ') : '';
        const msg = item.msg || 'Invalid value';
        return field ? `${field}: ${msg}` : msg;
      })
      .join('; ');
  }

  if (detail && typeof detail === 'object') {
    return detail.message || JSON.stringify(detail);
  }

  if (error.response?.data?.message) {
    return String(error.response.data.message);
  }

  if (error.message) {
    return error.message;
  }

  return fallback;
}
```

frontend/src/layouts/MainLayout.tsx 147 lines

```tsx
import React, { useState } from 'react';
import { NavLink, useNavigate } from 'react-router-dom';
import {
  LayoutDashboard, Users, HeartHandshake, Database,
  GitPullRequest, Calendar, Building2, BarChart3,
  ShieldCheck, Settings, LogOut, Menu, X, Activity
} from 'lucide-react';
import { useAuth } from '../context/AuthContext';

interface SidebarProps {
  children: React.ReactNode;
}

const navItems = [
  { name: 'Dashboard', path: '/', icon: LayoutDashboard },
  { name: 'Donors', path: '/donors', icon: Users },
  { name: 'Donations', path: '/donations', icon: HeartHandshake },
  { name: 'Blood Inventory', path: '/inventory', icon: Database },
  { name: 'Blood Requests', path: '/requests', icon: GitPullRequest },
  { name: 'Donation Events', path: '/events', icon: Calendar },
  { name: 'Hospitals', path: '/hospitals', icon: Building2 },
  { name: 'Reports', path: '/reports', icon: BarChart3 },
  { name: 'User Management', path: '/users', icon: ShieldCheck, adminOnly: true },
  { name: 'Settings', path: '/settings', icon: Settings },
];

export const MainLayout: React.FC<SidebarProps> = ({ children }) => {
  const [sidebarOpen, setSidebarOpen] = useState(false);
  const { user, logout, isAdmin } = useAuth();
  const navigate = useNavigate();

  return (
    <div className="flex h-screen bg-slate-50 overflow-hidden">
      {/* Mobile sidebar backdrop */}
      {sidebarOpen && (
        <div
          className="fixed inset-0 z-40 bg-slate-900/60 lg:hidden"
          onClick={() => setSidebarOpen(false)}
        />
      )}

      {/* Sidebar Navigation */}
      <aside className={`
        fixed inset-y-0 left-0 z-50 w-64 bg-slate-900 text-slate-200 flex flex-col transition-transfor
m duration-300 ease-in-out
        lg:static lg:translate-x-0
        ${sidebarOpen ? 'translate-x-0' : '-translate-x-full'}
      `}>
        {/* Brand Header */}
        <div className="flex items-center justify-between px-6 h-16 border-b border-slate-800 bg-slate
-950/40">
          <div className="flex items-center space-x-3 cursor-pointer" onClick={() => navigate('/')}>
            <div className="w-9 h-9 rounded-lg bg-red-700 flex items-center justify-center text-white
font-bold shadow-md shadow-red-900/30">
              
            </div>
            <div>
              <span className="font-bold text-white text-base tracking-tight leading-tight block">BDMS
</span>
```

```
              <span className="text-[10px] text-red-400 font-medium uppercase tracking-wider block">Blood Bank System</span>
            </div>
          </div>
          <button
            className="lg:hidden text-slate-400 hover:text-white"
            onClick={() => setSidebarOpen(false)}
          >
            <X className="w-5 h-5" />
          </button>
        </div>

        {/* Navigation Links */}
        <nav className="flex-1 px-3 py-4 space-y-1 overflow-y-auto">
          {navItems.map((item) => {
            if (item.adminOnly && !isAdmin) return null;
            const Icon = item.icon;
            return (
              <NavLink
                key={item.path}
                to={item.path}
                onClick={() => setSidebarOpen(false)}
                className={({ isActive }) => `
                  flex items-center px-3.5 py-2.5 rounded-lg text-sm font-medium transition-colors
                  ${isActive
                    ? 'bg-red-700 text-white shadow-sm shadow-red-950/40'
                    : 'text-slate-300 hover:bg-slate-800 hover:text-white'}
                `}
              >
                <Icon className="w-4 h-4 mr-3 shrink-0" />
                <span>{item.name}</span>
              </NavLink>
            );
          })}
        </nav>

        {/* User Profile and Logout */}
        <div className="p-3 border-t border-slate-800 bg-slate-950/30">
          <div className="flex items-center justify-between px-3 py-2 rounded-lg bg-slate-800/60">
            <div className="overflow-hidden pr-2">
              <p className="text-sm font-medium text-white truncate">{user?.full_name || 'Staff'}</p>
              <p className="text-xs text-red-400 capitalize">{user?.role || 'User'}</p>
            </div>
            <button
              onClick={logout}
              title="Logout"
              className="p-1.5 text-slate-400 hover:text-red-400 hover:bg-slate-700/60 rounded-md transition-colors"
            >
              <LogOut className="w-4 h-4" />
            </button>
          </div>
        </div>
      </aside>

      {/* Main Content Area */}
      <div className="flex-1 flex flex-col min-w-0 overflow-hidden">
        {/* Top Header */}
        <header className="h-16 bg-white border-b border-slate-200 flex items-center justify-between px-4 sm:px-6 z-10">
          <div className="flex items-center space-x-3">
            <button
              onClick={() => setSidebarOpen(true)}
```

```
              className="p-2 text-slate-600 hover:text-slate-900 hover:bg-slate-100 rounded-lg lg:hidd
en"
            >
              <Menu className="w-5 h-5" />
            </button>
            <div className="flex items-center space-x-2 text-xs font-medium text-slate-500">
              <span>Healthcare Administration</span>
              <span>/</span>
              <span className="text-slate-900 font-semibold">Active Session</span>
            </div>
          </div>

          <div className="flex items-center space-x-4">
            <div className="hidden sm:flex items-center space-x-1.5 text-xs text-emerald-700 bg-emeral
d-50 border border-emerald-200 px-2.5 py-1 rounded-full">
              <Activity className="w-3.5 h-3.5 text-emerald-600 animate-pulse" />
              <span className="font-medium">System Online</span>
            </div>
            <div className="text-xs font-semibold px-2 py-1 bg-slate-100 text-slate-700 rounded border
border-slate-200">
              MySQL 8.0 Connected
            </div>
          </div>
        </header>

        {/* Dynamic Page Views */}
        <main className="flex-1 overflow-y-auto p-4 sm:p-6 lg:p-8">
          <div className="max-w-7xl mx-auto">
            {children}
          </div>
        </main>
      </div>
    </div>
  );
};
```

frontend/src/pages/Login.tsx 111 lines

```tsx
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import apiClient from '../api/client';
import { useAuth } from '../context/AuthContext';
import { Droplet, Lock, Mail, AlertCircle } from 'lucide-react';

export const Login: React.FC = () => {
  const [email, setEmail] = useState('admin@bloodbank.org');
  const [password, setPassword] = useState('Admin@123');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setError(null);
    try {
      const response = await apiClient.post('/auth/login', { email, password });
      login(response.data);
      navigate('/');
    } catch (err: any) {
      setError(err.response?.data?.detail || 'Invalid email or password. Please verify credentials.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen bg-slate-900 flex flex-col justify-center items-center px-4 sm:px-6 l
g:px-8">
      <div className="sm:mx-auto sm:w-full sm:max-w-md">
        <div className="flex justify-center">
          <div className="w-14 h-14 rounded-2xl bg-red-700 flex items-center justify-center shadow-lg
shadow-red-900/40 text-white">
            <Droplet className="w-8 h-8 fill-current" />
          </div>
        </div>
        <h2 className="mt-4 text-center text-2xl font-bold tracking-tight text-white">
          Blood Donation Management System
        </h2>
        <p className="mt-1 text-center text-sm text-slate-400">
          Sign in to access your administrative dashboard
        </p>
      </div>

      <div className="mt-8 sm:mx-auto sm:w-full sm:max-w-md">
        <div className="bg-white py-8 px-6 shadow-xl rounded-xl sm:px-10 border border-slate-200">
          {error && (
            <div className="mb-4 bg-red-50 border border-red-200 text-red-700 text-sm p-3 rounded-lg f
lex items-center space-x-2">
              <AlertCircle className="w-4 h-4 shrink-0 text-red-600" />
              <span>{error}</span>
            </div>
          )}

          <form className="space-y-5" onSubmit={handleSubmit}>
            <div>
```

```
              <label className="block text-sm font-medium text-slate-700">Email address</label>
              <div className="mt-1 relative rounded-md shadow-sm">
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none t
ext-slate-400">
                  <Mail className="h-4 w-4" />
                </div>
                <input
                  type="email"
                  required
                  value={email}
                  onChange={(e) => setEmail(e.target.value)}
                  className="block w-full pl-10 pr-3 py-2 border border-slate-300 rounded-lg text-sm f
ocus:outline-none focus:ring-2 focus:ring-red-600 focus:border-red-600"
                  placeholder="admin@bloodbank.org"
                />
              </div>
            </div>

            <div>
              <label className="block text-sm font-medium text-slate-700">Password</label>
              <div className="mt-1 relative rounded-md shadow-sm">
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none t
ext-slate-400">
                  <Lock className="h-4 w-4" />
                </div>
                <input
                  type="password"
                  required
                  value={password}
                  onChange={(e) => setPassword(e.target.value)}
                  className="block w-full pl-10 pr-3 py-2 border border-slate-300 rounded-lg text-sm f
ocus:outline-none focus:ring-2 focus:ring-red-600 focus:border-red-600"
                  placeholder="••••••••"
                />
              </div>
            </div>

            <div>
              <button
                type="submit"
                disabled={loading}
                className="w-full flex justify-center py-2.5 px-4 border border-transparent rounded-lg
shadow-sm text-sm font-medium text-white bg-red-700 hover:bg-red-800 focus:outline-none focus:ring-2 focus:rin
g-offset-2 focus:ring-red-600 disabled:opacity-50 transition-colors"
              >
                {loading ? 'Authenticating...' : 'Sign In'}
              </button>
            </div>
          </form>

          <div className="mt-6 border-t border-slate-100 pt-4">
            <div className="text-xs text-slate-500 bg-slate-50 p-2.5 rounded border border-slate-200">
              <span className="font-semibold block text-slate-700">Demo System Credentials:</span>
              <span>Admin: admin@bloodbank.org / Admin@123</span>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
};
```

frontend/src/pages/Dashboard.tsx 280 lines

```tsx
import React, { useEffect, useState } from 'react';
import {
  Users, CheckCircle2, Database, AlertTriangle,
  HeartHandshake, PlusCircle, ArrowUpRight, Activity
} from 'lucide-react';
import {
  BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer, CartesianGrid, AreaChart, Area
} from 'recharts';
import apiClient from '../api/client';
import { DashboardStats } from '../types';
import { useNavigate } from 'react-router-dom';

export const Dashboard: React.FC = () => {
  const [stats, setStats] = useState<DashboardStats | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const navigate = useNavigate();

  useEffect(() => {
    fetchStats();
  }, []);

  const fetchStats = async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await apiClient.get('/dashboard/stats');
      setStats(res.data);
    } catch (err: any) {
      setError(err.response?.data?.detail || 'Unable to fetch dashboard metrics from MySQL.');
    } finally {
      setLoading(false);
    }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-red-700"></div>
        <span className="ml-3 text-sm text-slate-600">Loading database statistics...</span>
      </div>
    );
  }

  if (error || !stats) {
    return (
      <div className="p-6 bg-red-50 border border-red-200 rounded-xl text-red-800">
        <div className="flex items-center space-x-3">
          <AlertTriangle className="w-5 h-5 text-red-600 shrink-0" />
          <h3 className="text-base font-semibold">Backend / Database Connection Error</h3>
        </div>
        <p className="mt-2 text-sm text-red-700">{error || 'Failed to load data.'}</p>
        <button
          onClick={fetchStats}
          className="mt-4 px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-semibold hover:bg-red-800"
        >
          Retry Fetching
        </button>
```

```
    59        </div>
    60      );
    61    }
    62
    63    return (
    64      <div className="space-y-6">
    65        {/* Page Title & Quick Actions */}
    66        <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
    67          <div>
    68            <h1 className="text-2xl font-bold tracking-tight text-slate-900">Hospital Administration Das
hboard</h1>
    69            <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
    70              Real-time blood stock levels, voluntary donor records, and hospital demand analytics
    71            </p>
    72          </div>
    73          <div className="flex items-center space-x-2">
    74            <button
    75              onClick={() => navigate('/donors')}
    76              className="inline-flex items-center px-3 py-2 bg-white border border-slate-300 rounded-lg
text-xs font-medium text-slate-700 hover:bg-slate-50 shadow-sm transition"
    77            >
    78              <PlusCircle className="w-3.5 h-3.5 mr-1.5 text-slate-500" />
    79              Register Donor
    80            </button>
    81            <button
    82              onClick={() => navigate('/donations')}
    83              className="inline-flex items-center px-3 py-2 bg-red-700 text-white rounded-lg text-xs fon
t-medium hover:bg-red-800 shadow-sm transition"
    84            >
    85              <HeartHandshake className="w-3.5 h-3.5 mr-1.5 text-white" />
    86              Record Donation
    87            </button>
    88          </div>
    89        </div>
    90
    91        {/* KPI Cards */}
    92        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
    93          <div className="bg-white p-5 rounded-xl border border-slate-200 shadow-sm flex items-center ju
stify-between">
    94            <div>
    95              <p className="text-xs font-medium text-slate-500">Registered Donors</p>
    96              <p className="text-2xl font-bold text-slate-900 mt-1">{stats.total_donors}</p>
    97              <p className="text-xs text-emerald-600 mt-1 font-medium flex items-center">
    98                <CheckCircle2 className="w-3 h-3 mr-1" />
    99                {stats.eligible_donors} eligible donors
   100              </p>
   101            </div>
   102            <div className="w-12 h-12 bg-red-50 text-red-700 rounded-xl flex items-center justify-cente
r">
   103              <Users className="w-6 h-6" />
   104            </div>
   105          </div>
   106
   107          <div className="bg-white p-5 rounded-xl border border-slate-200 shadow-sm flex items-center ju
stify-between">
   108            <div>
   109              <p className="text-xs font-medium text-slate-500">Available Blood Units</p>
   110              <p className="text-2xl font-bold text-slate-900 mt-1">{stats.available_units}</p>
   111              <p className="text-xs text-slate-500 mt-1">Across 8 blood groups</p>
   112            </div>
   113            <div className="w-12 h-12 bg-blue-50 text-blue-700 rounded-xl flex items-center justify-cent
er">
   114              <Database className="w-6 h-6" />
```

```
            </div>
          </div>

          <div className="bg-white p-5 rounded-xl border border-slate-200 shadow-sm flex items-center ju
stify-between">
            <div>
              <p className="text-xs font-medium text-slate-500">Pending Requests</p>
              <p className="text-2xl font-bold text-amber-600 mt-1">{stats.pending_requests}</p>
              <p className="text-xs text-amber-600 mt-1 font-medium flex items-center">
                <Activity className="w-3 h-3 mr-1" />
                Urgent hospital orders
              </p>
            </div>
            <div className="w-12 h-12 bg-amber-50 text-amber-700 rounded-xl flex items-center justify-ce
nter">
              <AlertTriangle className="w-6 h-6" />
            </div>
          </div>

          <div className="bg-white p-5 rounded-xl border border-slate-200 shadow-sm flex items-center ju
stify-between">
            <div>
              <p className="text-xs font-medium text-slate-500">Donations This Month</p>
              <p className="text-2xl font-bold text-slate-900 mt-1">{stats.donations_this_month}</p>
              <p className="text-xs text-emerald-600 mt-1 font-medium">Verified completed units</p>
            </div>
            <div className="w-12 h-12 bg-emerald-50 text-emerald-700 rounded-xl flex items-center justif
y-center">
              <HeartHandshake className="w-6 h-6" />
            </div>
          </div>
        </div>

        {/* Visual Analytics Charts */}
        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
          {/* Inventory Distribution Chart */}
          <div className="bg-white p-5 rounded-xl border border-slate-200 shadow-sm">
            <div className="flex items-center justify-between mb-4">
              <h2 className="text-sm font-bold text-slate-900">Inventory Units by Blood Group</h2>
              <span className="text-xs font-semibold px-2 py-0.5 bg-slate-100 text-slate-600 rounded">
                Cold Store Batches
              </span>
            </div>
            <div className="h-64">
              <ResponsiveContainer width="100%" height="100%">
                <BarChart data={stats.inventory_by_group} margin={{ top: 10, right: 10, left: -20, botto
m: 0 }}>
                  <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#E2E8F0" />
                  <XAxis dataKey="blood_group" tickLine={false} tick={{ fill: '#64748B', fontSize: 12 }}
/>
                  <YAxis allowDecimals={false} tickLine={false} tick={{ fill: '#64748B', fontSize: 12 }}
/>
                  <Tooltip
                    contentStyle={{ backgroundColor: '#0F172A', borderRadius: '8px', color: '#FFF', font
Size: '12px' }}
                    itemStyle={{ color: '#F87171' }}
                  />
                  <Bar dataKey="units" fill="#B91C2C" radius={[4, 4, 0, 0]} />
                </BarChart>
              </ResponsiveContainer>
            </div>
          </div>

```

```
        {/* Monthly Donation Trends */}
        <div className="bg-white p-5 rounded-xl border border-slate-200 shadow-sm">
          <div className="flex items-center justify-between mb-4">
            <h2 className="text-sm font-bold text-slate-900">6-Month Donation Trends</h2>
            <span className="text-xs font-semibold px-2 py-0.5 bg-slate-100 text-slate-600 rounded">
              Trend Metrics
            </span>
          </div>
          <div className="h-64">
            {stats.monthly_trends.length > 0 ? (
              <ResponsiveContainer width="100%" height="100%">
                <AreaChart data={stats.monthly_trends} margin={{ top: 10, right: 10, left: -20, botto
m: 0 }}>
                  <defs>
                    <linearGradient id="colorDonations" x1="0" y1="0" x2="0" y2="1">
                      <stop offset="5%" stopColor="#B91C2C" stopOpacity={0.4}/>
                      <stop offset="95%" stopColor="#B91C2C" stopOpacity={0}/>
                    </linearGradient>
                  </defs>
                  <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#E2E8F0" />
                  <XAxis dataKey="month" tickLine={false} tick={{ fill: '#64748B', fontSize: 12 }} />
                  <YAxis allowDecimals={false} tickLine={false} tick={{ fill: '#64748B', fontSize: 12
}} />
                  <Tooltip
                    contentStyle={{ backgroundColor: '#0F172A', borderRadius: '8px', color: '#FFF', fo
ntSize: '12px' }}
                  />
                  <Area type="monotone" dataKey="donations" stroke="#B91C2C" strokeWidth={2} fillOpaci
ty={1} fill="url(#colorDonations)" />
                </AreaChart>
              </ResponsiveContainer>
            ) : (
              <div className="h-full flex items-center justify-center text-xs text-slate-400">
                No trend history yet. Record donations to populate this graph.
              </div>
            )}
          </div>
        </div>
      </div>

      {/* Tables Preview: Recent Donations & Hospital Requests */}
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Recent Donations */}
        <div className="bg-white rounded-xl border border-slate-200 shadow-sm overflow-hidden">
          <div className="px-5 py-4 border-b border-slate-100 flex items-center justify-between">
            <h3 className="text-sm font-bold text-slate-900">Recent Completed Donations</h3>
            <button
              onClick={() => navigate('/donations')}
              className="text-xs font-semibold text-red-700 hover:text-red-800 flex items-center"
            >
              View all <ArrowUpRight className="w-3.5 h-3.5 ml-0.5" />
            </button>
          </div>
          <div className="divide-y divide-slate-100">
            {stats.recent_donations.length > 0 ? (
              stats.recent_donations.map((d) => (
                <div key={d.id} className="px-5 py-3 flex items-center justify-between text-xs hover:b
g-slate-50">
                  <div>
                    <p className="font-semibold text-slate-800">{d.donor_name}</p>
                    <p className="text-slate-500">{d.donation_code} • {d.donation_date}</p>
                  </div>
                  <div className="text-right">
```

```
                    <span className="inline-block px-2 py-0.5 font-bold rounded bg-red-100 text-red-800">
                      {d.blood_group}
                    </span>
                    <p className="text-slate-500 text-[11px] mt-0.5">{d.status}</p>
                  </div>
                </div>
              ))
            ) : (
              <div className="p-6 text-center text-xs text-slate-400">No donations recorded yet.</div>
            )}
          </div>
        </div>

        {/* Latest Requests */}
        <div className="bg-white rounded-xl border border-slate-200 shadow-sm overflow-hidden">
          <div className="px-5 py-4 border-b border-slate-100 flex items-center justify-between">
            <h3 className="text-sm font-bold text-slate-900">Active Hospital Requests</h3>
            <button
              onClick={() => navigate('/requests')}
              className="text-xs font-semibold text-red-700 hover:text-red-800 flex items-center"
            >
              View all <ArrowUpRight className="w-3.5 h-3.5 ml-0.5" />
            </button>
          </div>
          <div className="divide-y divide-slate-100">
            {stats.latest_requests.length > 0 ? (
              stats.latest_requests.map((r) => (
                <div key={r.id} className="px-5 py-3 flex items-center justify-between text-xs hover:bg-slate-50">
                  <div>
                    <p className="font-semibold text-slate-800">{r.hospital_name}</p>
                    <p className="text-slate-500">{r.request_code} • Needed {r.required_date}</p>
                  </div>
                  <div className="text-right">
                    <span className={`inline-block px-2 py-0.5 font-bold rounded ${
                      r.urgency === 'Critical' ? 'bg-red-100 text-red-700' :
                      r.urgency === 'Urgent' ? 'bg-amber-100 text-amber-700' :
                      'bg-slate-100 text-slate-700'
                    }`}>
                      {r.blood_group} • {r.units_requested} Units
                    </span>
                    <p className="text-slate-500 text-[11px] mt-0.5">{r.status}</p>
                  </div>
                </div>
              ))
            ) : (
              <div className="p-6 text-center text-xs text-slate-400">No hospital requests found.</div>
            )}
          </div>
        </div>
      </div>
    </div>
  );
};
```

frontend/src/pages/Donors.tsx 333 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { Donor } from '../types';
import { Plus, Search, Check, X, AlertCircle } from 'lucide-react';
import { formatApiError } from '../utils/errorHandler';

export const Donors: React.FC = () => {
  const [donors, setDonors] = useState<Donor[]>([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [bloodGroup, setBloodGroup] = useState('');
  const [isEligible, setIsEligible] = useState<string>('');
  const [showModal, setShowModal] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [success, setSuccess] = useState<string | null>(null);

  // Form State
  const [formData, setFormData] = useState({
    full_name: '',
    email: '',
    phone: '',
    date_of_birth: '1995-01-01',
    gender: 'Male',
    blood_group: 'O+',
    address: '',
    city: '',
    is_eligible: true,
    notes: ''
  });

  useEffect(() => {
    fetchDonors();
  }, [bloodGroup, isEligible]);

  const fetchDonors = async () => {
    setLoading(true);
    try {
      const params: any = {};
      if (search) params.search = search;
      if (bloodGroup) params.blood_group = bloodGroup;
      if (isEligible !== '') params.is_eligible = isEligible === 'true';

      const res = await apiClient.get('/donors', { params });
      setDonors(res.data);
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const handleSearchSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    fetchDonors();
  };

  const handleCreateDonor = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
```

```
    setSuccess(null);
    try {
      const payload = {
        ...formData,
        email: formData.email.trim() ? formData.email.trim() : null,
        address: formData.address.trim() ? formData.address.trim() : null,
        notes: formData.notes.trim() ? formData.notes.trim() : null
      };
      const res = await apiClient.post('/donors', payload);
      setShowModal(false);
      setSuccess(`Donor ${res.data.full_name} (${res.data.donor_code}) registered successfully!`);
      setTimeout(() => setSuccess(null), 4000);
      setFormData({
        full_name: '',
        email: '',
        phone: '',
        date_of_birth: '1995-01-01',
        gender: 'Male',
        blood_group: 'O+',
        address: '',
        city: '',
        is_eligible: true,
        notes: ''
      });
      fetchDonors();
    } catch (err: any) {
      setError(formatApiError(err, 'Failed to register donor.'));
    }
  };

  return (
    <div className="space-y-6">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">Donor Management</h1>
          <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
            Register voluntary blood donors, inspect donation histories, and verify eligibility
          </p>
        </div>
        <button
          onClick={() => {
            setError(null);
            setShowModal(true);
          }}
          className="inline-flex items-center justify-center px-4 py-2 bg-red-700 text-white rounded-lg text-sm font-medium hover:bg-red-800 shadow-sm transition"
        >
          <Plus className="w-4 h-4 mr-1.5" />
          Add New Donor
        </button>
      </div>

      {success && (
        <div className="bg-emerald-50 border border-emerald-200 text-emerald-800 text-xs sm:text-sm p-3 rounded-lg flex items-center">
          <Check className="w-4 h-4 mr-2 text-emerald-600 shrink-0" />
          <span>{success}</span>
        </div>
      )}

      {/* Filter and Search Bar */}
      <div className="bg-white p-4 rounded-xl border border-slate-200 shadow-sm flex flex-col md:flex-row gap-3">
```

```
          <form onSubmit={handleSearchSubmit} className="flex-1 relative">
            <Search className="w-4 h-4 absolute left-3 top-3 text-slate-400" />
            <input
              type="text"
              placeholder="Search by name, donor code, phone, or city..."
              value={search}
              onChange={(e) => setSearch(e.target.value)}
              className="w-full pl-9 pr-4 py-2 border border-slate-300 rounded-lg text-sm focus:outline-
none focus:ring-2 focus:ring-red-600 focus:border-red-600"
            />
          </form>
          <div className="flex gap-2">
            <select
              value={bloodGroup}
              onChange={(e) => setBloodGroup(e.target.value)}
              className="border border-slate-300 rounded-lg px-3 py-2 text-sm bg-white focus:outline-non
e focus:ring-2 focus:ring-red-600"
            >
              <option value="">All Blood Groups</option>
              {['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'].map((bg) => (
                <option key={bg} value={bg}>{bg}</option>
              ))}
            </select>

            <select
              value={isEligible}
              onChange={(e) => setIsEligible(e.target.value)}
              className="border border-slate-300 rounded-lg px-3 py-2 text-sm bg-white focus:outline-non
e focus:ring-2 focus:ring-red-600"
            >
              <option value="">All Eligibility</option>
              <option value="true">Eligible Only</option>
              <option value="false">Deferred / Ineligible</option>
            </select>
          </div>
        </div>

        {/* Donors Table */}
        <div className="bg-white rounded-xl border border-slate-200 shadow-sm overflow-hidden">
          <div className="overflow-x-auto">
            <table className="w-full text-left text-xs sm:text-sm">
              <thead className="bg-slate-50 border-b border-slate-200 text-slate-600 font-semibold">
                <tr>
                  <th className="px-5 py-3.5">Code</th>
                  <th className="px-5 py-3.5">Donor Name</th>
                  <th className="px-5 py-3.5">Blood Group</th>
                  <th className="px-5 py-3.5">Phone / Contact</th>
                  <th className="px-5 py-3.5">City</th>
                  <th className="px-5 py-3.5">Eligibility</th>
                  <th className="px-5 py-3.5">Last Donated</th>
                </tr>
              </thead>
              <tbody className="divide-y divide-slate-100">
                {loading ? (
                  <tr>
                    <td colSpan={7} className="px-5 py-8 text-center text-slate-500">
                      Loading donors...
                    </td>
                  </tr>
                ) : donors.length === 0 ? (
                  <tr>
                    <td colSpan={7} className="px-5 py-8 text-center text-slate-500">
                      No donor records found matching criteria.
```

```
                  </td>
                </tr>
              ) : (
                donors.map((d) => (
                  <tr key={d.id} className="hover:bg-slate-50 transition">
                    <td className="px-5 py-3 font-mono font-medium text-slate-900">{d.donor_code}</td>
                    <td className="px-5 py-3 font-medium text-slate-900">{d.full_name}</td>
                    <td className="px-5 py-3">
                      <span className="inline-block px-2 py-0.5 font-bold rounded bg-red-100 text-red-
800 text-xs">
                        {d.blood_group}
                      </span>
                    </td>
                    <td className="px-5 py-3 text-slate-600">{d.phone}</td>
                    <td className="px-5 py-3 text-slate-600">{d.city}</td>
                    <td className="px-5 py-3">
                      {d.is_eligible ? (
                        <span className="inline-flex items-center px-2 py-0.5 rounded-full text-xs fon
t-medium bg-emerald-100 text-emerald-800">
                          <Check className="w-3 h-3 mr-1" /> Eligible
                        </span>
                      ) : (
                        <span className="inline-flex items-center px-2 py-0.5 rounded-full text-xs fon
t-medium bg-amber-100 text-amber-800">
                          <X className="w-3 h-3 mr-1" /> Deferred
                        </span>
                      )}
                    </td>
                    <td className="px-5 py-3 text-slate-500">
                      {d.last_donation_date || 'Never'}
                    </td>
                  </tr>
                ))
              )}
            </tbody>
          </table>
        </div>
      </div>

      {/* Modal Dialog for Registration */}
      {showModal && (
        <div className="fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 flex items-center justify-c
enter p-4">
          <div className="bg-white rounded-xl shadow-xl max-w-lg w-full p-6 border border-slate-200">
            <div className="flex justify-between items-center pb-3 border-b border-slate-100">
              <h3 className="text-base font-bold text-slate-900">Register Voluntary Donor</h3>
              <button onClick={() => setShowModal(false)} className="text-slate-400 hover:text-slate-6
00">
                <X className="w-5 h-5" />
              </button>
            </div>

            {error && (
              <div className="mt-3 bg-red-50 text-red-700 text-xs p-3 rounded flex items-center">
                <AlertCircle className="w-4 h-4 mr-2 shrink-0" />
                {error}
              </div>
            )}

            <form onSubmit={handleCreateDonor} className="mt-4 space-y-3.5 text-sm">
              <div className="grid grid-cols-2 gap-3">
                <div className="col-span-2">
                  <label className="block text-xs font-semibold text-slate-700">Full Name *</label>
```

```
                  <input
                    type="text"
                    required
                    value={formData.full_name}
                    onChange={(e) => setFormData({ ...formData, full_name: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Email Address (Optiona
l)</label>
                  <input
                    type="email"
                    value={formData.email}
                    onChange={(e) => setFormData({ ...formData, email: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                    placeholder="donor@example.com"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Phone Number *</label>
                  <input
                    type="text"
                    required
                    value={formData.phone}
                    onChange={(e) => setFormData({ ...formData, phone: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Blood Group *</label>
                  <select
                    value={formData.blood_group}
                    onChange={(e) => setFormData({ ...formData, blood_group: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-whit
e"
                  >
                    {['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'].map((bg) => (
                      <option key={bg} value={bg}>{bg}</option>
                    ))}
                  </select>
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Date of Birth *</label
>
                  <input
                    type="date"
                    required
                    value={formData.date_of_birth}
                    onChange={(e) => setFormData({ ...formData, date_of_birth: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Gender *</label>
                  <select
                    value={formData.gender}
                    onChange={(e) => setFormData({ ...formData, gender: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-whit
e"
                  >
                    <option value="Male">Male</option>
                    <option value="Female">Female</option>
```

```
                    <option value="Other">Other</option>
                  </select>
                </div>
                <div className="col-span-2">
                  <label className="block text-xs font-semibold text-slate-700">City *</label>
                  <input
                    type="text"
                    required
                    value={formData.city}
                    onChange={(e) => setFormData({ ...formData, city: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
              </div>

              <div className="flex justify-end space-x-2 pt-4 border-t border-slate-100">
                <button
                  type="button"
                  onClick={() => setShowModal(false)}
                  className="px-4 py-2 border border-slate-300 rounded-lg text-xs font-medium text-sla
te-700 hover:bg-slate-50"
                >
                  Cancel
                </button>
                <button
                  type="submit"
                  className="px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-medium hover:bg-r
ed-800"
                >
                  Save Donor
                </button>
              </div>
            </form>
          </div>
        </div>
      )}
    </div>
  );
};
```

frontend/src/pages/Donations.tsx 296 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { Donation, Donor } from '../types';
import { Plus, Check, AlertCircle, X } from 'lucide-react';
import { formatApiError } from '../utils/errorHandler';

export const Donations: React.FC = () => {
  const [donations, setDonations] = useState<Donation[]>([]);
  const [donors, setDonors] = useState<Donor[]>([]);
  const [loading, setLoading] = useState(true);
  const [showModal, setShowModal] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [formData, setFormData] = useState({
    donor_id: 1,
    donation_date: new Date().toISOString().split('T')[0],
    blood_group: 'O+',
    component_type: 'Whole Blood',
    units: 1,
    hemoglobin_g_dl: 14.0,
    blood_pressure: '120/80',
    pulse_rate: 72,
    screening_status: 'Passed',
    status: 'Completed',
    remarks: 'Routine center voluntary donation'
  });

  useEffect(() => {
    fetchDonations();
    fetchDonorsList();
  }, []);

  const fetchDonations = async () => {
    setLoading(true);
    try {
      const res = await apiClient.get('/donations');
      setDonations(res.data);
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const fetchDonorsList = async () => {
    try {
      const res = await apiClient.get('/donors?limit=100');
      setDonors(res.data);
      if (res.data.length > 0) {
        setFormData((prev) => ({
          ...prev,
          donor_id: res.data[0].id,
          blood_group: res.data[0].blood_group
        }));
      }
    } catch (err: any) {
      console.error(err);
    }
  };
```

```
  const handleDonorSelect = (donorId: number) => {
    const selected = donors.find((d) => d.id === donorId);
    setFormData((prev) => ({
      ...prev,
      donor_id: donorId,
      blood_group: selected ? selected.blood_group : prev.blood_group
    }));
  };

  const handleCreateDonation = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
    try {
      await apiClient.post('/donations', formData);
      setShowModal(false);
      fetchDonations();
    } catch (err: any) {
      setError(formatApiError(err, 'Failed to record donation.'));
    }
  };

  return (
    <div className="space-y-6">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">Donation Management</h1>
          <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
            Log medical donor vitals, screenings, and generate cold-chain inventory batches
          </p>
        </div>
        <button
          onClick={() => setShowModal(true)}
          className="inline-flex items-center justify-center px-4 py-2 bg-red-700 text-white rounded-l
g text-sm font-medium hover:bg-red-800 shadow-sm transition"
        >
          <Plus className="w-4 h-4 mr-1.5" />
          Record Donation
        </button>
      </div>

      {/* Donations Table */}
      <div className="bg-white rounded-xl border border-slate-200 shadow-sm overflow-hidden">
        <div className="overflow-x-auto">
          <table className="w-full text-left text-xs sm:text-sm">
            <thead className="bg-slate-50 border-b border-slate-200 text-slate-600 font-semibold">
              <tr>
                <th className="px-5 py-3.5">Donation Code</th>
                <th className="px-5 py-3.5">Donor</th>
                <th className="px-5 py-3.5">Blood Group</th>
                <th className="px-5 py-3.5">Component</th>
                <th className="px-5 py-3.5">Date</th>
                <th className="px-5 py-3.5">Screening</th>
                <th className="px-5 py-3.5">Status</th>
                <th className="px-5 py-3.5">Inventory</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-slate-100">
              {loading ? (
                <tr>
                  <td colSpan={8} className="px-5 py-8 text-center text-slate-500">
                    Loading donation history...
                  </td>
```

```
                </tr>
              ) : donations.length === 0 ? (
                <tr>
                  <td colSpan={8} className="px-5 py-8 text-center text-slate-500">
                    No donation records logged.
                  </td>
                </tr>
              ) : (
                donations.map((d) => (
                  <tr key={d.id} className="hover:bg-slate-50 transition">
                    <td className="px-5 py-3 font-mono font-medium text-slate-900">{d.donation_code}</
td>
                    <td className="px-5 py-3 font-medium text-slate-900">{d.donor_name || `Donor #${d.
donor_id}`}</td>
                    <td className="px-5 py-3">
                      <span className="inline-block px-2 py-0.5 font-bold rounded bg-red-100 text-red-
800 text-xs">
                        {d.blood_group}
                      </span>
                    </td>
                    <td className="px-5 py-3 text-slate-600">{d.component_type}</td>
                    <td className="px-5 py-3 text-slate-600">{d.donation_date}</td>
                    <td className="px-5 py-3">
                      <span className={`inline-flex items-center px-2 py-0.5 rounded text-xs font-semi
bold ${
                        d.screening_status === 'Passed' ? 'bg-emerald-100 text-emerald-800' : 'bg-red-
100 text-red-800'
                      }`}>
                        {d.screening_status}
                      </span>
                    </td>
                    <td className="px-5 py-3">
                      <span className={`inline-flex items-center px-2 py-0.5 rounded text-xs font-semi
bold ${
                        d.status === 'Completed' ? 'bg-blue-100 text-blue-800' : 'bg-amber-100 text-am
ber-800'
                      }`}>
                        {d.status}
                      </span>
                    </td>
                    <td className="px-5 py-3">
                      {d.inventory_generated ? (
                        <span className="text-emerald-700 font-semibold text-xs flex items-center">
                          <Check className="w-3.5 h-3.5 mr-1" /> Added to Stock
                        </span>
                      ) : (
                        <span className="text-slate-400 text-xs">Pending Batch</span>
                      )}
                    </td>
                  </tr>
                ))
              )}
            </tbody>
          </table>
        </div>
      </div>

      {/* Record Donation Modal */}
      {showModal && (
        <div className="fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 flex items-center justify-c
enter p-4">
          <div className="bg-white rounded-xl shadow-xl max-w-lg w-full p-6 border border-slate-200">
            <div className="flex justify-between items-center pb-3 border-b border-slate-100">
```

```
              <h3 className="text-base font-bold text-slate-900">Record Voluntary Donation</h3>
              <button onClick={() => setShowModal(false)} className="text-slate-400 hover:text-slate-6
00">
                <X className="w-5 h-5" />
              </button>
            </div>

            {error && (
              <div className="mt-3 bg-red-50 text-red-700 text-xs p-3 rounded flex items-center">
                <AlertCircle className="w-4 h-4 mr-2 shrink-0" />
                {error}
              </div>
            )}

            <form onSubmit={handleCreateDonation} className="mt-4 space-y-3.5 text-sm">
              <div>
                <label className="block text-xs font-semibold text-slate-700">Select Donor *</label>
                <select
                  value={formData.donor_id}
                  onChange={(e) => handleDonorSelect(Number(e.target.value))}
                  className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-white"
                >
                  {donors.map((d) => (
                    <option key={d.id} value={d.id}>
                      {d.full_name} ({d.blood_group}) - {d.phone}
                    </option>
                  ))}
                </select>
              </div>

              <div className="grid grid-cols-2 gap-3">
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Component *</label>
                  <select
                    value={formData.component_type}
                    onChange={(e) => setFormData({ ...formData, component_type: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-whit
e"
                  >
                    <option value="Whole Blood">Whole Blood</option>
                    <option value="Red Blood Cells">Red Blood Cells</option>
                    <option value="Platelets">Platelets</option>
                    <option value="Plasma">Plasma</option>
                  </select>
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Donation Date *</label
>
                  <input
                    type="date"
                    required
                    value={formData.donation_date}
                    onChange={(e) => setFormData({ ...formData, donation_date: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Hemoglobin (g/dL)</lab
el>
                  <input
                    type="number"
                    step="0.1"
                    value={formData.hemoglobin_g_dl}
```

```
                    onChange={(e) => setFormData({ ...formData, hemoglobin_g_dl: Number(e.target.valu
e) })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Blood Pressure</label>
                  <input
                    type="text"
                    value={formData.blood_pressure}
                    onChange={(e) => setFormData({ ...formData, blood_pressure: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                    placeholder="120/80"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Screening Outcome</lab
el>
                  <select
                    value={formData.screening_status}
                    onChange={(e) => setFormData({ ...formData, screening_status: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-whit
e"
                  >
                    <option value="Passed">Passed (Eligible)</option>
                    <option value="Deferred">Deferred</option>
                    <option value="Failed">Failed</option>
                  </select>
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Status</label>
                  <select
                    value={formData.status}
                    onChange={(e) => setFormData({ ...formData, status: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-whit
e"
                  >
                    <option value="Completed">Completed (Auto-generate Batch)</option>
                    <option value="Scheduled">Scheduled</option>
                  </select>
                </div>
              </div>

              <div className="flex justify-end space-x-2 pt-4 border-t border-slate-100">
                <button
                  type="button"
                  onClick={() => setShowModal(false)}
                  className="px-4 py-2 border border-slate-300 rounded-lg text-xs font-medium text-sla
te-700 hover:bg-slate-50"
                >
                  Cancel
                </button>
                <button
                  type="submit"
                  className="px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-medium hover:bg-r
ed-800"
                >
                  Confirm & Save
                </button>
              </div>
            </form>
          </div>
        </div>
```

```
293         )}
294       </div>
295     );
296   };
```

frontend/src/pages/Inventory.tsx 168 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { BloodInventoryItem } from '../types';
import { Clock } from 'lucide-react';

export const Inventory: React.FC = () => {
  const [inventory, setInventory] = useState<BloodInventoryItem[]>([]);
  const [summary, setSummary] = useState<any>(null);
  const [loading, setLoading] = useState(true);
  const [bloodGroup, setBloodGroup] = useState('');
  const [status, setStatus] = useState('Available');

  useEffect(() => {
    fetchInventory();
    fetchSummary();
  }, [bloodGroup, status]);

  const fetchInventory = async () => {
    setLoading(true);
    try {
      const params: any = {};
      if (bloodGroup) params.blood_group = bloodGroup;
      if (status) params.status = status;
      const res = await apiClient.get('/inventory', { params });
      setInventory(res.data);
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const fetchSummary = async () => {
    try {
      const res = await apiClient.get('/inventory/summary');
      setSummary(res.data);
    } catch (err: any) {
      console.error(err);
    }
  };

  return (
    <div className="space-y-6">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">Blood Inventory & Cold Stor
age</h1>
          <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
            Traceable blood units with FIFO earliest-expiry allocation and real-time low-stock trackin
g
          </p>
        </div>
      </div>

      {/* Stock Cards by Blood Group */}
      {summary && (
        <div className="grid grid-cols-2 sm:grid-cols-4 lg:grid-cols-8 gap-3">
          {summary.groups.map((g: any) => (
            <div
```

```
              key={g.blood_group}
              className={`p-3 rounded-xl border text-center ${
                g.is_low_stock
                  ? 'bg-amber-50/60 border-amber-200 text-amber-900'
                  : 'bg-white border-slate-200 text-slate-900'
              }`}
            >
              <span className="text-sm font-black text-red-700 block">{g.blood_group}</span>
              <span className="text-xl font-bold block mt-1">{g.units_available}</span>
              <span className="text-[10px] uppercase font-semibold text-slate-500 tracking-wider">
                {g.is_low_stock ? 'Low Stock' : 'Units'}
              </span>
            </div>
          ))}
        </div>
      )}

      {/* Filter Toolbar */}
      <div className="bg-white p-4 rounded-xl border border-slate-200 shadow-sm flex flex-wrap gap-3">
        <select
          value={bloodGroup}
          onChange={(e) => setBloodGroup(e.target.value)}
          className="border border-slate-300 rounded-lg px-3 py-2 text-sm bg-white focus:outline-none
focus:ring-2 focus:ring-red-600"
        >
          <option value="">All Blood Groups</option>
          {['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'].map((bg) => (
            <option key={bg} value={bg}>{bg}</option>
          ))}
        </select>

        <select
          value={status}
          onChange={(e) => setStatus(e.target.value)}
          className="border border-slate-300 rounded-lg px-3 py-2 text-sm bg-white focus:outline-none
focus:ring-2 focus:ring-red-600"
        >
          <option value="">All Statuses</option>
          <option value="Available">Available (Unissued)</option>
          <option value="Issued">Issued</option>
          <option value="Expired">Expired</option>
        </select>

        {summary && summary.expiring_soon_count > 0 && (
          <div className="ml-auto flex items-center text-xs font-semibold text-amber-700 bg-amber-50 b
order border-amber-200 px-3 py-1.5 rounded-lg">
            <Clock className="w-3.5 h-3.5 mr-1.5" />
            {summary.expiring_soon_count} Units Expiring Within 7 Days
          </div>
        )}
      </div>

      {/* Inventory Batches Table */}
      <div className="bg-white rounded-xl border border-slate-200 shadow-sm overflow-hidden">
        <div className="overflow-x-auto">
          <table className="w-full text-left text-xs sm:text-sm">
            <thead className="bg-slate-50 border-b border-slate-200 text-slate-600 font-semibold">
              <tr>
                <th className="px-5 py-3.5">Batch Number</th>
                <th className="px-5 py-3.5">Blood Group</th>
                <th className="px-5 py-3.5">Component</th>
                <th className="px-5 py-3.5">Volume</th>
                <th className="px-5 py-3.5">Collection Date</th>
```

```
                <th className="px-5 py-3.5">Expiry Date</th>
                <th className="px-5 py-3.5">Location</th>
                <th className="px-5 py-3.5">Status</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-slate-100">
              {loading ? (
                <tr>
                  <td colSpan={8} className="px-5 py-8 text-center text-slate-500">
                    Loading inventory records...
                  </td>
                </tr>
              ) : inventory.length === 0 ? (
                <tr>
                  <td colSpan={8} className="px-5 py-8 text-center text-slate-500">
                    No inventory units found in database.
                  </td>
                </tr>
              ) : (
                inventory.map((item) => (
                  <tr key={item.id} className="hover:bg-slate-50 transition">
                    <td className="px-5 py-3 font-mono font-medium text-slate-900">{item.batch_number}
</td>
                    <td className="px-5 py-3">
                      <span className="inline-block px-2 py-0.5 font-bold rounded bg-red-100 text-red-
800 text-xs">
                        {item.blood_group}
                      </span>
                    </td>
                    <td className="px-5 py-3 text-slate-600">{item.component_type}</td>
                    <td className="px-5 py-3 text-slate-600">{item.volume_ml} mL</td>
                    <td className="px-5 py-3 text-slate-600">{item.collection_date}</td>
                    <td className="px-5 py-3 font-semibold text-slate-800">{item.expiry_date}</td>
                    <td className="px-5 py-3 text-slate-500">{item.storage_location}</td>
                    <td className="px-5 py-3">
                      <span className={`inline-flex items-center px-2 py-0.5 rounded text-xs font-semi
bold ${
                        item.status === 'Available' ? 'bg-emerald-100 text-emerald-800' :
                        item.status === 'Issued' ? 'bg-blue-100 text-blue-800' :
                        'bg-red-100 text-red-800'
                      }`}>
                        {item.status}
                      </span>
                    </td>
                  </tr>
                ))
              )}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  );
};
```

frontend/src/pages/Requests.tsx 391 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { BloodRequest, Hospital } from '../types';
import { Plus, Send, AlertCircle, X, ShieldAlert } from 'lucide-react';
import { formatApiError } from '../utils/errorHandler';

export const Requests: React.FC = () => {
  const [requests, setRequests] = useState<BloodRequest[]>([]);
  const [hospitals, setHospitals] = useState<Hospital[]>([]);
  const [loading, setLoading] = useState(true);
  const [showCreateModal, setShowCreateModal] = useState(false);
  const [showIssueModal, setShowIssueModal] = useState(false);
  const [selectedRequest, setSelectedRequest] = useState<BloodRequest | null>(null);
  const [issueQty, setIssueQty] = useState(1);
  const [recipientName, setRecipientName] = useState('');
  const [error, setError] = useState<string | null>(null);

  const [formData, setFormData] = useState({
    hospital_id: 1,
    patient_name: '',
    blood_group: 'O+',
    component_type: 'Whole Blood',
    units_requested: 2,
    urgency: 'Normal',
    required_date: new Date().toISOString().split('T')[0],
    notes: ''
  });

  useEffect(() => {
    fetchRequests();
    fetchHospitals();
  }, []);

  const fetchRequests = async () => {
    setLoading(true);
    try {
      const res = await apiClient.get('/requests');
      setRequests(res.data);
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const fetchHospitals = async () => {
    try {
      const res = await apiClient.get('/hospitals');
      setHospitals(res.data);
      if (res.data.length > 0) {
        setFormData((prev) => ({ ...prev, hospital_id: res.data[0].id }));
      }
    } catch (err: any) {
      console.error(err);
    }
  };

  const handleCreateRequest = async (e: React.FormEvent) => {
    e.preventDefault();
```

```
    setError(null);
    try {
      await apiClient.post('/requests', formData);
      setShowCreateModal(false);
      fetchRequests();
    } catch (err: any) {
      setError(formatApiError(err, 'Failed to submit blood request.'));
    }
  };

  const openIssueModal = (req: BloodRequest) => {
    setSelectedRequest(req);
    const unfulfilled = req.units_requested - req.units_fulfilled;
    setIssueQty(unfulfilled > 0 ? unfulfilled : 1);
    setRecipientName('');
    setError(null);
    setShowIssueModal(true);
  };

  const handleIssueSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!selectedRequest) return;
    setError(null);
    try {
      await apiClient.post('/issues', {
        request_id: selectedRequest.id,
        units_to_issue: Number(issueQty),
        recipient_name: recipientName,
        notes: `Issued to hospital staff: ${recipientName}`
      });
      setShowIssueModal(false);
      fetchRequests();
    } catch (err: any) {
      setError(formatApiError(err, 'Blood issue transaction failed.'));
    }
  };

  return (
    <div className="space-y-6">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">Hospital Blood Requests & I
ssuing</h1>
          <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
            Process orders, approve units, and issue batch-traceable blood directly to hospitals
          </p>
        </div>
        <button
          onClick={() => setShowCreateModal(true)}
          className="inline-flex items-center justify-center px-4 py-2 bg-red-700 text-white rounded-l
g text-sm font-medium hover:bg-red-800 shadow-sm transition"
        >
          <Plus className="w-4 h-4 mr-1.5" />
          Create Blood Request
        </button>
      </div>

      {/* Requests Table */}
      <div className="bg-white rounded-xl border border-slate-200 shadow-sm overflow-hidden">
        <div className="overflow-x-auto">
          <table className="w-full text-left text-xs sm:text-sm">
            <thead className="bg-slate-50 border-b border-slate-200 text-slate-600 font-semibold">
              <tr>
```

```
                <th className="px-5 py-3.5">Code</th>
                <th className="px-5 py-3.5">Hospital</th>
                <th className="px-5 py-3.5">Patient</th>
                <th className="px-5 py-3.5">Blood Group</th>
                <th className="px-5 py-3.5">Requested / Fulfilled</th>
                <th className="px-5 py-3.5">Urgency</th>
                <th className="px-5 py-3.5">Required Date</th>
                <th className="px-5 py-3.5">Status</th>
                <th className="px-5 py-3.5 text-right">Action</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-slate-100">
              {loading ? (
                <tr>
                  <td colSpan={9} className="px-5 py-8 text-center text-slate-500">
                    Loading blood requests...
                  </td>
                </tr>
              ) : requests.length === 0 ? (
                <tr>
                  <td colSpan={9} className="px-5 py-8 text-center text-slate-500">
                    No active hospital blood requests.
                  </td>
                </tr>
              ) : (
                requests.map((r) => {
                  const isFulfilled = r.units_fulfilled >= r.units_requested;
                  return (
                    <tr key={r.id} className="hover:bg-slate-50 transition">
                      <td className="px-5 py-3 font-mono font-medium text-slate-900">{r.request_code}
</td>
                      <td className="px-5 py-3 font-medium text-slate-900">{r.hospital_name}</td>
                      <td className="px-5 py-3 text-slate-600">{r.patient_name || 'N/A'}</td>
                      <td className="px-5 py-3">
                        <span className="inline-block px-2 py-0.5 font-bold rounded bg-red-100 text-re
d-800 text-xs">
                          {r.blood_group}
                        </span>
                      </td>
                      <td className="px-5 py-3 font-semibold text-slate-800">
                        {r.units_fulfilled} / {r.units_requested} Units
                      </td>
                      <td className="px-5 py-3">
                        <span className={`inline-flex items-center px-2 py-0.5 rounded text-xs font-se
mibold ${
                          r.urgency === 'Critical' ? 'bg-red-100 text-red-800' :
                          r.urgency === 'Urgent' ? 'bg-amber-100 text-amber-800' :
                          'bg-slate-100 text-slate-700'
                        }`}>
                          {r.urgency}
                        </span>
                      </td>
                      <td className="px-5 py-3 text-slate-600">{r.required_date}</td>
                      <td className="px-5 py-3">
                        <span className={`inline-flex items-center px-2 py-0.5 rounded text-xs font-se
mibold ${
                          r.status === 'Fulfilled' ? 'bg-emerald-100 text-emerald-800' :
                          r.status === 'Partially Fulfilled' ? 'bg-blue-100 text-blue-800' :
                          'bg-amber-100 text-amber-800'
                        }`}>
                          {r.status}
                        </span>
                      </td>
```

```
                      <td className="px-5 py-3 text-right">
                        {!isFulfilled && (
                          <button
                            onClick={() => openIssueModal(r)}
                            className="inline-flex items-center px-3 py-1 bg-red-700 hover:bg-red-800
text-white rounded text-xs font-semibold transition"
                          >
                            <Send className="w-3 h-3 mr-1" />
                            Issue Blood
                          </button>
                        )}
                      </td>
                    </tr>
                  );
                })
              )}
            </tbody>
          </table>
        </div>
      </div>

      {/* Create Request Modal */}
      {showCreateModal && (
        <div className="fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 flex items-center justify-c
enter p-4">
          <div className="bg-white rounded-xl shadow-xl max-w-lg w-full p-6 border border-slate-200">
            <div className="flex justify-between items-center pb-3 border-b border-slate-100">
              <h3 className="text-base font-bold text-slate-900">New Hospital Blood Request</h3>
              <button onClick={() => setShowCreateModal(false)} className="text-slate-400 hover:text-s
late-600">
                <X className="w-5 h-5" />
              </button>
            </div>

            {error && (
              <div className="mt-3 bg-red-50 text-red-700 text-xs p-3 rounded flex items-center">
                <AlertCircle className="w-4 h-4 mr-2 shrink-0" />
                {error}
              </div>
            )}

            <form onSubmit={handleCreateRequest} className="mt-4 space-y-3.5 text-sm">
              <div>
                <label className="block text-xs font-semibold text-slate-700">Requesting Hospital *</l
abel>
                <select
                  value={formData.hospital_id}
                  onChange={(e) => setFormData({ ...formData, hospital_id: Number(e.target.value) })}
                  className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-white"
                >
                  {hospitals.map((h) => (
                    <option key={h.id} value={h.id}>
                      {h.name} - {h.city}
                    </option>
                  ))}
                </select>
              </div>

              <div className="grid grid-cols-2 gap-3">
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Blood Group *</label>
                  <select
                    value={formData.blood_group}
```

```
                    onChange={(e) => setFormData({ ...formData, blood_group: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-white"
                  >
                    {['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'].map((bg) => (
                      <option key={bg} value={bg}>{bg}</option>
                    ))}
                  </select>
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Units Needed *</label>
                  <input
                    type="number"
                    min="1"
                    required
                    value={formData.units_requested}
                    onChange={(e) => setFormData({ ...formData, units_requested: Number(e.target.value) })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Urgency Level</label>
                  <select
                    value={formData.urgency}
                    onChange={(e) => setFormData({ ...formData, urgency: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-white"
                  >
                    <option value="Normal">Normal</option>
                    <option value="Urgent">Urgent</option>
                    <option value="Critical">Critical</option>
                  </select>
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Required Date *</label>
                  <input
                    type="date"
                    required
                    value={formData.required_date}
                    onChange={(e) => setFormData({ ...formData, required_date: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div className="col-span-2">
                  <label className="block text-xs font-semibold text-slate-700">Patient Name (Optional)</label>
                  <input
                    type="text"
                    value={formData.patient_name}
                    onChange={(e) => setFormData({ ...formData, patient_name: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                    placeholder="e.g. John Doe (ICU)"
                  />
                </div>
              </div>

              <div className="flex justify-end space-x-2 pt-4 border-t border-slate-100">
                <button
                  type="button"
                  onClick={() => setShowCreateModal(false)}
                  className="px-4 py-2 border border-slate-300 rounded-lg text-xs font-medium text-sla
```

```
te-700 hover:bg-slate-50"
                  >
                    Cancel
                  </button>
                  <button
                    type="submit"
                    className="px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-medium hover:bg-r
ed-800"
                  >
                    Submit Request
                  </button>
                </div>
              </form>
            </div>
          </div>
        )}

        {/* Transactional Blood Issue Modal */}
        {showIssueModal && selectedRequest && (
          <div className="fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 flex items-center justify-c
enter p-4">
            <div className="bg-white rounded-xl shadow-xl max-w-md w-full p-6 border border-slate-200">
              <div className="flex justify-between items-center pb-3 border-b border-slate-100">
                <h3 className="text-base font-bold text-slate-900">Authorize Blood Issue</h3>
                <button onClick={() => setShowIssueModal(false)} className="text-slate-400 hover:text-sl
ate-600">
                  <X className="w-5 h-5" />
                </button>
              </div>

              {error && (
                <div className="mt-3 bg-red-50 text-red-700 text-xs p-3 rounded flex items-center">
                  <AlertCircle className="w-4 h-4 mr-2 shrink-0" />
                  {error}
                </div>
              )}

              <div className="my-3 p-3 bg-slate-50 rounded border border-slate-200 text-xs text-slate-70
0 space-y-1">
                <p><span className="font-semibold">Request:</span> {selectedRequest.request_code}</p>
                <p><span className="font-semibold">Hospital:</span> {selectedRequest.hospital_name}</p>
                <p><span className="font-semibold">Blood Group:</span> {selectedRequest.blood_group} ({s
electedRequest.component_type})</p>
                <p><span className="font-semibold">Unfulfilled Balance:</span> {selectedRequest.units_re
quested - selectedRequest.units_fulfilled} Units</p>
              </div>

              <form onSubmit={handleIssueSubmit} className="space-y-3.5 text-sm">
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Units to Issue Now *</la
bel>
                  <input
                    type="number"
                    min="1"
                    max={selectedRequest.units_requested - selectedRequest.units_fulfilled}
                    required
                    value={issueQty}
                    onChange={(e) => setIssueQty(Number(e.target.value))}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>

                <div>
```

```
                  <label className="block text-xs font-semibold text-slate-700">Authorized Recipient / Nurse Name *</label>
                  <input
                    type="text"
                    required
                    placeholder="e.g. Nurse Sarah (Trauma Desk)"
                    value={recipientName}
                    onChange={(e) => setRecipientName(e.target.value)}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>

                <div className="p-2.5 bg-amber-50 border border-amber-200 rounded text-[11px] text-amber-800 flex items-start">
                  <ShieldAlert className="w-4 h-4 mr-1.5 shrink-0 text-amber-600 mt-0.5" />
                  <span>
                    ACID Transaction: Batches will be locked and deducted atomically based on earliest expiration date.
                  </span>
                </div>

                <div className="flex justify-end space-x-2 pt-4 border-t border-slate-100">
                  <button
                    type="button"
                    onClick={() => setShowIssueModal(false)}
                    className="px-4 py-2 border border-slate-300 rounded-lg text-xs font-medium text-slate-700 hover:bg-slate-50"
                  >
                    Cancel
                  </button>
                  <button
                    type="submit"
                    className="px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-medium hover:bg-red-800"
                  >
                    Confirm Issue
                  </button>
                </div>
              </form>
            </div>
          </div>
        )}
      </div>
    );
  };
```

frontend/src/pages/Events.tsx 328 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { DonationEvent, Donor } from '../types';
import { Plus, Calendar, MapPin, Users, X, AlertCircle } from 'lucide-react';
import { formatApiError } from '../utils/errorHandler';

export const Events: React.FC = () => {
  const [events, setEvents] = useState<DonationEvent[]>([]);
  const [donors, setDonors] = useState<Donor[]>([]);
  const [loading, setLoading] = useState(true);
  const [showEventModal, setShowEventModal] = useState(false);
  const [showRegModal, setShowRegModal] = useState(false);
  const [selectedEventId, setSelectedEventId] = useState<number | null>(null);
  const [selectedDonorId, setSelectedDonorId] = useState<number | null>(null);
  const [error, setError] = useState<string | null>(null);

  const [eventForm, setEventForm] = useState({
    title: '',
    organizer: '',
    venue: '',
    city: '',
    event_date: new Date().toISOString().split('T')[0],
    start_time: '09:00:00',
    end_time: '16:00:00',
    capacity: 100,
    status: 'Upcoming',
    description: ''
  });

  useEffect(() => {
    fetchEvents();
    fetchDonors();
  }, []);

  const fetchEvents = async () => {
    setLoading(true);
    try {
      const res = await apiClient.get('/events');
      setEvents(res.data);
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const fetchDonors = async () => {
    try {
      const res = await apiClient.get('/donors?limit=100');
      setDonors(res.data);
      if (res.data.length > 0) setSelectedDonorId(res.data[0].id);
    } catch (err: any) {
      console.error(err);
    }
  };

  const handleCreateEvent = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
```

```
    try {
      await apiClient.post('/events', eventForm);
      setShowEventModal(false);
      fetchEvents();
    } catch (err: any) {
      setError(formatApiError(err, 'Failed to schedule event.'));
    }
  };

  const openRegisterModal = (eventId: number) => {
    setSelectedEventId(eventId);
    setError(null);
    setShowRegModal(true);
  };

  const handleRegisterDonor = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!selectedEventId || !selectedDonorId) return;
    setError(null);
    try {
      await apiClient.post('/events/register', {
        event_id: selectedEventId,
        donor_id: selectedDonorId
      });
      setShowRegModal(false);
      fetchEvents();
    } catch (err: any) {
      setError(formatApiError(err, 'Failed to register donor.'));
    }
  };

  return (
    <div className="space-y-6">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">Donation Camps & Community
Events</h1>
          <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
            Organize mobile blood drives, manage venue capacities, and coordinate donor pre-registrati
ons
          </p>
        </div>
        <button
          onClick={() => setShowEventModal(true)}
          className="inline-flex items-center justify-center px-4 py-2 bg-red-700 text-white rounded-l
g text-sm font-medium hover:bg-red-800 shadow-sm transition"
        >
          <Plus className="w-4 h-4 mr-1.5" />
          Schedule Event
        </button>
      </div>

      {/* Events Grid */}
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {loading ? (
          <div className="col-span-full text-center py-12 text-slate-500">Loading events...</div>
        ) : events.length === 0 ? (
          <div className="col-span-full text-center py-12 text-slate-500">No scheduled donation event
s.</div>
        ) : (
          events.map((evt) => (
            <div key={evt.id} className="bg-white rounded-xl border border-slate-200 shadow-sm p-5 fle
x flex-col justify-between">
```

```
                <div>
                  <div className="flex justify-between items-start">
                    <span className="font-mono text-xs text-slate-500 font-semibold">{evt.event_code}</s
pan>
                    <span className={`px-2 py-0.5 rounded text-xs font-semibold ${
                      evt.status === 'Completed' ? 'bg-slate-100 text-slate-700' :
                      evt.status === 'Ongoing' ? 'bg-emerald-100 text-emerald-800 animate-pulse' :
                      'bg-blue-100 text-blue-800'
                    }`}>
                      {evt.status}
                    </span>
                  </div>
                  <h3 className="font-bold text-slate-900 text-base mt-2">{evt.title}</h3>
                  <p className="text-xs text-red-700 font-medium mt-0.5">Organizer: {evt.organizer}</p>

                  <div className="mt-4 space-y-2 text-xs text-slate-600">
                    <div className="flex items-center">
                      <Calendar className="w-3.5 h-3.5 mr-2 text-slate-400" />
                      <span>{evt.event_date} ({evt.start_time.slice(0, 5)} - {evt.end_time.slice(0, 5)})
</span>
                    </div>
                    <div className="flex items-center">
                      <MapPin className="w-3.5 h-3.5 mr-2 text-slate-400" />
                      <span className="truncate">{evt.venue}, {evt.city}</span>
                    </div>
                    <div className="flex items-center">
                      <Users className="w-3.5 h-3.5 mr-2 text-slate-400" />
                      <span>{evt.registered_count} / {evt.capacity} Registered Participants</span>
                    </div>
                  </div>

                  {evt.description && (
                    <p className="mt-3 text-xs text-slate-500 bg-slate-50 p-2.5 rounded border border-sl
ate-100 line-clamp-2">
                      {evt.description}
                    </p>
                  )}
                </div>

                <div className="mt-5 pt-3 border-t border-slate-100 flex justify-end">
                  <button
                    onClick={() => openRegisterModal(evt.id)}
                    disabled={evt.status === 'Completed' || evt.registered_count >= evt.capacity}
                    className="px-3 py-1.5 bg-slate-900 hover:bg-slate-800 disabled:opacity-40 text-whit
e rounded text-xs font-medium transition"
                  >
                    Register Donor
                  </button>
                </div>
              </div>
            ))
          )}
        </div>

        {/* Schedule Event Modal */}
        {showEventModal && (
          <div className="fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 flex items-center justify-c
enter p-4">
            <div className="bg-white rounded-xl shadow-xl max-w-lg w-full p-6 border border-slate-200">
              <div className="flex justify-between items-center pb-3 border-b border-slate-100">
                <h3 className="text-base font-bold text-slate-900">Schedule Donation Drive</h3>
                <button onClick={() => setShowEventModal(false)} className="text-slate-400 hover:text-sl
ate-600">
```

```
                <X className="w-5 h-5" />
              </button>
            </div>

            {error && (
              <div className="mt-3 bg-red-50 text-red-700 text-xs p-3 rounded flex items-center">
                <AlertCircle className="w-4 h-4 mr-2 shrink-0" />
                {error}
              </div>
            )}

            <form onSubmit={handleCreateEvent} className="mt-4 space-y-3.5 text-sm">
              <div>
                <label className="block text-xs font-semibold text-slate-700">Event Title *</label>
                <input
                  type="text"
                  required
                  placeholder="e.g. City Tech Park Voluntary Blood Drive"
                  value={eventForm.title}
                  onChange={(e) => setEventForm({ ...eventForm, title: e.target.value })}
                  className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                />
              </div>

              <div className="grid grid-cols-2 gap-3">
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Organizer Name *</labe
l>
                  <input
                    type="text"
                    required
                    value={eventForm.organizer}
                    onChange={(e) => setEventForm({ ...eventForm, organizer: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">City *</label>
                  <input
                    type="text"
                    required
                    value={eventForm.city}
                    onChange={(e) => setEventForm({ ...eventForm, city: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div className="col-span-2">
                  <label className="block text-xs font-semibold text-slate-700">Venue / Location *</la
bel>
                  <input
                    type="text"
                    required
                    value={eventForm.venue}
                    onChange={(e) => setEventForm({ ...eventForm, venue: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Date *</label>
                  <input
                    type="date"
                    required
                    value={eventForm.event_date}
```

```
                    onChange={(e) => setEventForm({ ...eventForm, event_date: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Capacity *</label>
                  <input
                    type="number"
                    min="10"
                    required
                    value={eventForm.capacity}
                    onChange={(e) => setEventForm({ ...eventForm, capacity: Number(e.target.value) })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
              </div>

              <div className="flex justify-end space-x-2 pt-4 border-t border-slate-100">
                <button
                  type="button"
                  onClick={() => setShowEventModal(false)}
                  className="px-4 py-2 border border-slate-300 rounded-lg text-xs font-medium text-sla
te-700 hover:bg-slate-50"
                >
                  Cancel
                </button>
                <button
                  type="submit"
                  className="px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-medium hover:bg-r
ed-800"
                >
                  Create Event
                </button>
              </div>
            </form>
          </div>
        </div>
      )}

      {/* Register Donor Modal */}
      {showRegModal && (
        <div className="fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 flex items-center justify-c
enter p-4">
          <div className="bg-white rounded-xl shadow-xl max-w-sm w-full p-6 border border-slate-200">
            <div className="flex justify-between items-center pb-3 border-b border-slate-100">
              <h3 className="text-base font-bold text-slate-900">Pre-Register Donor</h3>
              <button onClick={() => setShowRegModal(false)} className="text-slate-400 hover:text-slat
e-600">
                <X className="w-5 h-5" />
              </button>
            </div>

            {error && (
              <div className="mt-3 bg-red-50 text-red-700 text-xs p-3 rounded flex items-center">
                <AlertCircle className="w-4 h-4 mr-2 shrink-0" />
                {error}
              </div>
            )}

            <form onSubmit={handleRegisterDonor} className="mt-4 space-y-4 text-sm">
              <div>
                <label className="block text-xs font-semibold text-slate-700">Select Donor</label>
                <select
```

```
                    value={selectedDonorId || ''}
                    onChange={(e) => setSelectedDonorId(Number(e.target.value))}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-white"
                  >
                    {donors.map((d) => (
                      <option key={d.id} value={d.id}>
                        {d.full_name} ({d.blood_group})
                      </option>
                    ))}
                  </select>
                </div>

                <div className="flex justify-end space-x-2 pt-3 border-t border-slate-100">
                  <button
                    type="button"
                    onClick={() => setShowRegModal(false)}
                    className="px-4 py-2 border border-slate-300 rounded-lg text-xs font-medium text-sla
te-700 hover:bg-slate-50"
                  >
                    Cancel
                  </button>
                  <button
                    type="submit"
                    className="px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-medium hover:bg-r
ed-800"
                  >
                    Confirm Registration
                  </button>
                </div>
              </form>
            </div>
          </div>
        )}
      </div>
    );
  };
```

frontend/src/pages/Hospitals.tsx 231 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { Hospital } from '../types';
import { Plus, Phone, Mail, MapPin, X, AlertCircle } from 'lucide-react';
import { formatApiError } from '../utils/errorHandler';

export const Hospitals: React.FC = () => {
  const [hospitals, setHospitals] = useState<Hospital[]>([]);
  const [loading, setLoading] = useState(true);
  const [showModal, setShowModal] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [formData, setFormData] = useState({
    name: '',
    license_number: '',
    contact_person: '',
    phone: '',
    email: '',
    address: '',
    city: ''
  });

  useEffect(() => {
    fetchHospitals();
  }, []);

  const fetchHospitals = async () => {
    setLoading(true);
    try {
      const res = await apiClient.get('/hospitals');
      setHospitals(res.data);
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const handleCreateHospital = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
    try {
      await apiClient.post('/hospitals', formData);
      setShowModal(false);
      setFormData({
        name: '',
        license_number: '',
        contact_person: '',
        phone: '',
        email: '',
        address: '',
        city: ''
      });
      fetchHospitals();
    } catch (err: any) {
      setError(formatApiError(err, 'Failed to register hospital.'));
    }
  };

```

```
    return (
      <div className="space-y-6">
        <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
          <div>
            <h1 className="text-2xl font-bold tracking-tight text-slate-900">Partner Hospitals & Medical Centers</h1>
            <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
              Registered medical trauma networks authorized to place emergency whole blood and component requests
            </p>
          </div>
          <button
            onClick={() => setShowModal(true)}
            className="inline-flex items-center justify-center px-4 py-2 bg-red-700 text-white rounded-lg text-sm font-medium hover:bg-red-800 shadow-sm transition"
          >
            <Plus className="w-4 h-4 mr-1.5" />
            Add Hospital
          </button>
        </div>

        {/* Hospital Cards Grid */}
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
          {loading ? (
            <div className="col-span-full text-center py-12 text-slate-500">Loading partner hospitals...</div>
          ) : hospitals.length === 0 ? (
            <div className="col-span-full text-center py-12 text-slate-500">No partner hospitals found.</div>
          ) : (
            hospitals.map((h) => (
              <div key={h.id} className="bg-white rounded-xl border border-slate-200 shadow-sm p-5 flex flex-col justify-between">
                <div>
                  <div className="flex justify-between items-start">
                    <span className="font-mono text-xs text-red-700 font-bold bg-red-50 px-2 py-0.5 rounded border border-red-100">
                      {h.hospital_code}
                    </span>
                    <span className="text-[11px] text-slate-500 font-medium">Lic: {h.license_number}</span>
                  </div>

                  <h3 className="font-bold text-slate-900 text-base mt-2">{h.name}</h3>
                  <p className="text-xs text-slate-500 font-medium">Contact: {h.contact_person}</p>

                  <div className="mt-4 space-y-2 text-xs text-slate-600">
                    <div className="flex items-center">
                      <Phone className="w-3.5 h-3.5 mr-2 text-slate-400" />
                      <span>{h.phone}</span>
                    </div>
                    <div className="flex items-center">
                      <Mail className="w-3.5 h-3.5 mr-2 text-slate-400" />
                      <span className="truncate">{h.email}</span>
                    </div>
                    <div className="flex items-center">
                      <MapPin className="w-3.5 h-3.5 mr-2 text-slate-400" />
                      <span className="truncate">{h.address}, {h.city}</span>
                    </div>
                  </div>
                </div>
              </div>
            ))
```

```
          )}
        </div>

        {/* Add Hospital Modal */}
        {showModal && (
          <div className="fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 flex items-center justify-center p-4">
            <div className="bg-white rounded-xl shadow-xl max-w-lg w-full p-6 border border-slate-200">
              <div className="flex justify-between items-center pb-3 border-b border-slate-100">
                <h3 className="text-base font-bold text-slate-900">Add Partner Hospital</h3>
                <button onClick={() => setShowModal(false)} className="text-slate-400 hover:text-slate-600">
                  <X className="w-5 h-5" />
                </button>
              </div>

              {error && (
                <div className="mt-3 bg-red-50 text-red-700 text-xs p-3 rounded flex items-center">
                  <AlertCircle className="w-4 h-4 mr-2 shrink-0" />
                  {error}
                </div>
              )}

              <form onSubmit={handleCreateHospital} className="mt-4 space-y-3.5 text-sm">
                <div className="grid grid-cols-2 gap-3">
                  <div className="col-span-2">
                    <label className="block text-xs font-semibold text-slate-700">Hospital / Facility Name *</label>
                    <input
                      type="text"
                      required
                      value={formData.name}
                      onChange={(e) => setFormData({ ...formData, name: e.target.value })}
                      className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                    />
                  </div>
                  <div>
                    <label className="block text-xs font-semibold text-slate-700">License Number *</label>
                    <input
                      type="text"
                      required
                      value={formData.license_number}
                      onChange={(e) => setFormData({ ...formData, license_number: e.target.value })}
                      className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                    />
                  </div>
                  <div>
                    <label className="block text-xs font-semibold text-slate-700">Contact Officer *</label>
                    <input
                      type="text"
                      required
                      value={formData.contact_person}
                      onChange={(e) => setFormData({ ...formData, contact_person: e.target.value })}
                      className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                    />
                  </div>
                  <div>
                    <label className="block text-xs font-semibold text-slate-700">Direct Phone *</label>
                    <input
                      type="text"
                      required
```

```
                    value={formData.phone}
                    onChange={(e) => setFormData({ ...formData, phone: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div>
                  <label className="block text-xs font-semibold text-slate-700">Email Address *</label
>
                  <input
                    type="email"
                    required
                    value={formData.email}
                    onChange={(e) => setFormData({ ...formData, email: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div className="col-span-2">
                  <label className="block text-xs font-semibold text-slate-700">Address *</label>
                  <input
                    type="text"
                    required
                    value={formData.address}
                    onChange={(e) => setFormData({ ...formData, address: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
                <div className="col-span-2">
                  <label className="block text-xs font-semibold text-slate-700">City *</label>
                  <input
                    type="text"
                    required
                    value={formData.city}
                    onChange={(e) => setFormData({ ...formData, city: e.target.value })}
                    className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
                  />
                </div>
              </div>

              <div className="flex justify-end space-x-2 pt-4 border-t border-slate-100">
                <button
                  type="button"
                  onClick={() => setShowModal(false)}
                  className="px-4 py-2 border border-slate-300 rounded-lg text-xs font-medium text-sla
te-700 hover:bg-slate-50"
                >
                  Cancel
                </button>
                <button
                  type="submit"
                  className="px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-medium hover:bg-r
ed-800"
                >
                  Register Hospital
                </button>
              </div>
            </form>
          </div>
        </div>
      )}
    </div>
  );
};
```

frontend/src/pages/Reports.tsx 239 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { Download, BarChart2, Building, Clock } from 'lucide-react';

export const Reports: React.FC = () => {
  const [activeTab, setActiveTab] = useState<'inventory' | 'hospitals' | 'expiring'>('inventory');
  const [inventoryData, setInventoryData] = useState<any[]>([]);
  const [hospitalData, setHospitalData] = useState<any[]>([]);
  const [expiringData, setExpiringData] = useState<any[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchReportData();
  }, [activeTab]);

  const fetchReportData = async () => {
    setLoading(true);
    try {
      if (activeTab === 'inventory') {
        const res = await apiClient.get('/reports/inventory-breakdown');
        setInventoryData(res.data);
      } else if (activeTab === 'hospitals') {
        const res = await apiClient.get('/reports/hospital-fulfillment');
        setHospitalData(res.data);
      } else if (activeTab === 'expiring') {
        const res = await apiClient.get('/reports/expiring-soon?days=14');
        setExpiringData(res.data);
      }
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const exportCSV = () => {
    let csvContent = "data:text/csv;charset=utf-8,";
    if (activeTab === 'inventory') {
      csvContent += "Blood Group,Component,Status,Units,Earliest Expiry\n";
      inventoryData.forEach(row => {
        csvContent += `${row.blood_group},${row.component_type},${row.status},${row.unit_count},${row.
earliest_expiry || ''}\n`;
      });
    } else if (activeTab === 'hospitals') {
      csvContent += "Hospital Name,City,Requests,Total Requested,Total Fulfilled,Fulfillment %\n";
      hospitalData.forEach(row => {
        csvContent += `"${row.hospital_name}",${row.city},${row.total_requests},${row.total_requeste
d},${row.total_fulfilled},${row.fulfillment_rate_percent}%\n`;
      });
    } else {
      csvContent += "Batch Number,Blood Group,Component,Days Remaining,Expiry Date,Storage\n";
      expiringData.forEach(row => {
        csvContent += `${row.batch_number},${row.blood_group},${row.component_type},${row.days_remaini
ng},${row.expiry_date},"${row.storage_location}"\n`;
      });
    }
    const encodedUri = encodeURI(csvContent);
    const link = document.createElement("a");
    link.setAttribute("href", encodedUri);
```

```
    link.setAttribute("download", `bdms_report_${activeTab}.csv`);
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  };

  return (
    <div className="space-y-6">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">Database Analytics & Report
s</h1>
          <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
            SQL aggregation summaries, clinical fulfillment audits, and CSV reporting exports
          </p>
        </div>
        <button
          onClick={exportCSV}
          className="inline-flex items-center justify-center px-4 py-2 bg-slate-900 text-white rounded
-lg text-sm font-medium hover:bg-slate-800 shadow-sm transition"
        >
          <Download className="w-4 h-4 mr-1.5" />
          Export CSV Report
        </button>
      </div>

      {/* Tabs */}
      <div className="flex space-x-2 border-b border-slate-200">
        <button
          onClick={() => setActiveTab('inventory')}
          className={`flex items-center px-4 py-2.5 text-xs font-semibold border-b-2 transition ${
            activeTab === 'inventory'
              ? 'border-red-700 text-red-700'
              : 'border-transparent text-slate-500 hover:text-slate-800'
          }`}
        >
          <BarChart2 className="w-4 h-4 mr-1.5" />
          Inventory Component Breakdown
        </button>
        <button
          onClick={() => setActiveTab('hospitals')}
          className={`flex items-center px-4 py-2.5 text-xs font-semibold border-b-2 transition ${
            activeTab === 'hospitals'
              ? 'border-red-700 text-red-700'
              : 'border-transparent text-slate-500 hover:text-slate-800'
          }`}
        >
          <Building className="w-4 h-4 mr-1.5" />
          Hospital Demand & Fulfillment
        </button>
        <button
          onClick={() => setActiveTab('expiring')}
          className={`flex items-center px-4 py-2.5 text-xs font-semibold border-b-2 transition ${
            activeTab === 'expiring'
              ? 'border-red-700 text-red-700'
              : 'border-transparent text-slate-500 hover:text-slate-800'
          }`}
        >
          <Clock className="w-4 h-4 mr-1.5" />
          Expiring Inventory (14 Days)
        </button>
      </div>

```

```
        {/* Report Table */}
        <div className="bg-white rounded-xl border border-slate-200 shadow-sm overflow-hidden">
          <div className="overflow-x-auto">
            {activeTab === 'inventory' && (
              <table className="w-full text-left text-xs sm:text-sm">
                <thead className="bg-slate-50 border-b border-slate-200 text-slate-600 font-semibold">
                  <tr>
                    <th className="px-5 py-3.5">Blood Group</th>
                    <th className="px-5 py-3.5">Component Type</th>
                    <th className="px-5 py-3.5">Status</th>
                    <th className="px-5 py-3.5">Batch Count</th>
                    <th className="px-5 py-3.5">Earliest Expiry</th>
                    <th className="px-5 py-3.5">Latest Expiry</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-slate-100">
                  {loading ? (
                    <tr><td colSpan={6} className="text-center py-8 text-slate-500">Loading report...</t
d></tr>
                  ) : (
                    inventoryData.map((row, i) => (
                      <tr key={i} className="hover:bg-slate-50">
                        <td className="px-5 py-3 font-bold text-red-700">{row.blood_group}</td>
                        <td className="px-5 py-3 text-slate-700">{row.component_type}</td>
                        <td className="px-5 py-3">
                          <span className="px-2 py-0.5 rounded text-xs font-semibold bg-slate-100 text-s
late-800">
                            {row.status}
                          </span>
                        </td>
                        <td className="px-5 py-3 font-semibold text-slate-900">{row.unit_count} Units</t
d>
                        <td className="px-5 py-3 text-slate-600">{row.earliest_expiry || 'N/A'}</td>
                        <td className="px-5 py-3 text-slate-600">{row.latest_expiry || 'N/A'}</td>
                      </tr>
                    ))
                  )}
                </tbody>
              </table>
            )}

            {activeTab === 'hospitals' && (
              <table className="w-full text-left text-xs sm:text-sm">
                <thead className="bg-slate-50 border-b border-slate-200 text-slate-600 font-semibold">
                  <tr>
                    <th className="px-5 py-3.5">Hospital Name</th>
                    <th className="px-5 py-3.5">City</th>
                    <th className="px-5 py-3.5">Total Orders</th>
                    <th className="px-5 py-3.5">Units Requested</th>
                    <th className="px-5 py-3.5">Units Fulfilled</th>
                    <th className="px-5 py-3.5">Fulfillment Rate</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-slate-100">
                  {loading ? (
                    <tr><td colSpan={6} className="text-center py-8 text-slate-500">Loading report...</t
d></tr>
                  ) : (
                    hospitalData.map((row, i) => (
                      <tr key={i} className="hover:bg-slate-50">
                        <td className="px-5 py-3 font-semibold text-slate-900">{row.hospital_name}</td>
                        <td className="px-5 py-3 text-slate-600">{row.city}</td>
                        <td className="px-5 py-3 text-slate-700">{row.total_requests}</td>
```

```
                        <td className="px-5 py-3 text-slate-700">{row.total_requested}</td>
                        <td className="px-5 py-3 font-semibold text-emerald-700">{row.total_fulfilled}</
td>
                        <td className="px-5 py-3">
                          <div className="flex items-center space-x-2">
                            <div className="w-24 bg-slate-100 rounded-full h-2 overflow-hidden">
                              <div
                                className="bg-emerald-600 h-2 rounded-full"
                                style={{ width: `${Math.min(row.fulfillment_rate_percent, 100)}%` }}
                              />
                            </div>
                            <span className="text-xs font-semibold text-slate-700">{row.fulfillment_rate
_percent}%</span>
                          </div>
                        </td>
                      </tr>
                    ))
                  )}
                </tbody>
              </table>
            )}

            {activeTab === 'expiring' && (
              <table className="w-full text-left text-xs sm:text-sm">
                <thead className="bg-slate-50 border-b border-slate-200 text-slate-600 font-semibold">
                  <tr>
                    <th className="px-5 py-3.5">Batch Number</th>
                    <th className="px-5 py-3.5">Blood Group</th>
                    <th className="px-5 py-3.5">Component</th>
                    <th className="px-5 py-3.5">Volume</th>
                    <th className="px-5 py-3.5">Days Remaining</th>
                    <th className="px-5 py-3.5">Expiry Date</th>
                    <th className="px-5 py-3.5">Location</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-slate-100">
                  {loading ? (
                    <tr><td colSpan={7} className="text-center py-8 text-slate-500">Loading report...</t
d></tr>
                  ) : expiringData.length === 0 ? (
                    <tr><td colSpan={7} className="text-center py-8 text-slate-500">No stock batches exp
iring within 14 days.</td></tr>
                  ) : (
                    expiringData.map((row, i) => (
                      <tr key={i} className="hover:bg-slate-50">
                        <td className="px-5 py-3 font-mono font-medium text-slate-900">{row.batch_numbe
r}</td>
                        <td className="px-5 py-3 font-bold text-red-700">{row.blood_group}</td>
                        <td className="px-5 py-3 text-slate-700">{row.component_type}</td>
                        <td className="px-5 py-3 text-slate-600">{row.volume_ml} mL</td>
                        <td className="px-5 py-3">
                          <span className="px-2 py-0.5 rounded text-xs font-bold bg-amber-100 text-amber
-900">
                            {row.days_remaining} Days
                          </span>
                        </td>
                        <td className="px-5 py-3 font-semibold text-slate-800">{row.expiry_date}</td>
                        <td className="px-5 py-3 text-slate-500">{row.storage_location}</td>
                      </tr>
                    ))
                  )}
                </tbody>
              </table>
```

```
          )}
        </div>
      </div>
    </div>
  );
};
```

frontend/src/pages/Users.tsx 199 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { User } from '../types';
import { Plus, X, AlertCircle } from 'lucide-react';
import { formatApiError } from '../utils/errorHandler';

export const UsersPage: React.FC = () => {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);
  const [showModal, setShowModal] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [formData, setFormData] = useState({
    full_name: '',
    email: '',
    password: '',
    role: 'staff',
    is_active: true
  });

  useEffect(() => {
    fetchUsers();
  }, []);

  const fetchUsers = async () => {
    setLoading(true);
    try {
      const res = await apiClient.get('/users');
      setUsers(res.data);
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const handleCreateUser = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
    try {
      await apiClient.post('/users', formData);
      setShowModal(false);
      setFormData({
        full_name: '',
        email: '',
        password: '',
        role: 'staff',
        is_active: true
      });
      fetchUsers();
    } catch (err: any) {
      setError(formatApiError(err, 'Failed to provision user.'));
    }
  };

  return (
    <div className="space-y-6">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
```

```
            <h1 className="text-2xl font-bold tracking-tight text-slate-900">User Management & Role Perm
issions</h1>
            <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
              Administer system personnel, assign staff/admin roles, and enforce security policies
            </p>
          </div>
          <button
            onClick={() => setShowModal(true)}
            className="inline-flex items-center justify-center px-4 py-2 bg-red-700 text-white rounded-l
g text-sm font-medium hover:bg-red-800 shadow-sm transition"
          >
            <Plus className="w-4 h-4 mr-1.5" />
            Add User
          </button>
        </div>

        <div className="bg-white rounded-xl border border-slate-200 shadow-sm overflow-hidden">
          <div className="overflow-x-auto">
            <table className="w-full text-left text-xs sm:text-sm">
              <thead className="bg-slate-50 border-b border-slate-200 text-slate-600 font-semibold">
                <tr>
                  <th className="px-5 py-3.5">Full Name</th>
                  <th className="px-5 py-3.5">Email Address</th>
                  <th className="px-5 py-3.5">Role</th>
                  <th className="px-5 py-3.5">Status</th>
                  <th className="px-5 py-3.5">Account Created</th>
                </tr>
              </thead>
              <tbody className="divide-y divide-slate-100">
                {loading ? (
                  <tr><td colSpan={5} className="text-center py-8 text-slate-500">Loading user account
s...</td></tr>
                ) : (
                  users.map((u) => (
                    <tr key={u.id} className="hover:bg-slate-50 transition">
                      <td className="px-5 py-3 font-semibold text-slate-900">{u.full_name}</td>
                      <td className="px-5 py-3 text-slate-600">{u.email}</td>
                      <td className="px-5 py-3">
                        <span className={`inline-flex items-center px-2 py-0.5 rounded text-xs font-bold
uppercase ${
                          u.role === 'admin' ? 'bg-red-100 text-red-800' : 'bg-slate-100 text-slate-700'
                        }`}>
                          {u.role}
                        </span>
                      </td>
                      <td className="px-5 py-3">
                        <span className={`inline-block px-2 py-0.5 rounded text-xs font-semibold ${
                          u.is_active ? 'bg-emerald-100 text-emerald-800' : 'bg-slate-200 text-slate-60
0'
                        }`}>
                          {u.is_active ? 'Active' : 'Deactivated'}
                        </span>
                      </td>
                      <td className="px-5 py-3 text-slate-500">{new Date(u.created_at).toLocaleDateStrin
g()}</td>
                    </tr>
                  ))
                )}
              </tbody>
            </table>
          </div>
        </div>

```

```
117         {showModal && (
118           <div className="fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 flex items-center justify-c
enter p-4">
119             <div className="bg-white rounded-xl shadow-xl max-w-md w-full p-6 border border-slate-200">
120               <div className="flex justify-between items-center pb-3 border-b border-slate-100">
121                 <h3 className="text-base font-bold text-slate-900">Provision User Account</h3>
122                 <button onClick={() => setShowModal(false)} className="text-slate-400 hover:text-slate-6
00">
123                   <X className="w-5 h-5" />
124                 </button>
125               </div>
126
127               {error && (
128                 <div className="mt-3 bg-red-50 text-red-700 text-xs p-3 rounded flex items-center">
129                   <AlertCircle className="w-4 h-4 mr-2 shrink-0" />
130                   {error}
131                 </div>
132               )}
133
134               <form onSubmit={handleCreateUser} className="mt-4 space-y-3.5 text-sm">
135                 <div>
136                   <label className="block text-xs font-semibold text-slate-700">Full Name *</label>
137                   <input
138                     type="text"
139                     required
140                     value={formData.full_name}
141                     onChange={(e) => setFormData({ ...formData, full_name: e.target.value })}
142                     className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
143                   />
144                 </div>
145                 <div>
146                   <label className="block text-xs font-semibold text-slate-700">Email Address *</label>
147                   <input
148                     type="email"
149                     required
150                     value={formData.email}
151                     onChange={(e) => setFormData({ ...formData, email: e.target.value })}
152                     className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
153                   />
154                 </div>
155                 <div>
156                   <label className="block text-xs font-semibold text-slate-700">Secure Password (min 6 c
hars) *</label>
157                   <input
158                     type="password"
159                     required
160                     minLength={6}
161                     value={formData.password}
162                     onChange={(e) => setFormData({ ...formData, password: e.target.value })}
163                     className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm"
164                   />
165                 </div>
166                 <div>
167                   <label className="block text-xs font-semibold text-slate-700">Role *</label>
168                   <select
169                     value={formData.role}
170                     onChange={(e) => setFormData({ ...formData, role: e.target.value })}
171                     className="mt-1 w-full border border-slate-300 rounded px-3 py-1.5 text-sm bg-white"
172                   >
173                     <option value="staff">Staff (Operational Access)</option>
174                     <option value="admin">Admin (Full Control)</option>
175                   </select>
176                 </div>
```

```
                <div className="flex justify-end space-x-2 pt-4 border-t border-slate-100">
                  <button
                    type="button"
                    onClick={() => setShowModal(false)}
                    className="px-4 py-2 border border-slate-300 rounded-lg text-xs font-medium text-sla
te-700 hover:bg-slate-50"
                  >
                    Cancel
                  </button>
                  <button
                    type="submit"
                    className="px-4 py-2 bg-red-700 text-white rounded-lg text-xs font-medium hover:bg-r
ed-800"
                  >
                    Create User
                  </button>
                </div>
              </form>
            </div>
          </div>
        )}
      </div>
    );
  };
```

frontend/src/pages/Settings.tsx 112 lines

```tsx
import React, { useState, useEffect } from 'react';
import apiClient from '../api/client';
import { SystemSetting } from '../types';
import { Save, Check, AlertCircle } from 'lucide-react';
import { useAuth } from '../context/AuthContext';
import { formatApiError } from '../utils/errorHandler';

export const SettingsPage: React.FC = () => {
  const [settings, setSettings] = useState<SystemSetting[]>([]);
  const [loading, setLoading] = useState(true);
  const [success, setSuccess] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const { isAdmin } = useAuth();

  useEffect(() => {
    fetchSettings();
  }, []);

  const fetchSettings = async () => {
    setLoading(true);
    try {
      const res = await apiClient.get('/settings');
      setSettings(res.data);
    } catch (err: any) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const handleUpdate = async (key: string, value: string) => {
    if (!isAdmin) {
      setError('Only administrators can modify system settings.');
      return;
    }
    setError(null);
    setSuccess(null);
    try {
      await apiClient.put(`/settings/${key}`, { setting_value: value });
      setSuccess(`Setting '${key}' updated successfully in MySQL!`);
      setTimeout(() => setSuccess(null), 3000);
      fetchSettings();
    } catch (err: any) {
      setError(formatApiError(err, 'Failed to update setting.'));
    }
  };

  return (
    <div className="space-y-6">
      <div>
        <h1 className="text-2xl font-bold tracking-tight text-slate-900">System Configuration & Operational Rules</h1>
        <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
          Configure real low-stock warning thresholds, deferral donation gaps, and shelf life parameters
        </p>
      </div>

      {success && (
```

```
        <div className="bg-emerald-50 border border-emerald-200 text-emerald-800 text-sm p-3 rounded-l
g flex items-center">
          <Check className="w-4 h-4 mr-2 text-emerald-600" />
          {success}
        </div>
      )}

      {error && (
        <div className="bg-red-50 border border-red-200 text-red-800 text-sm p-3 rounded-lg flex items
-center">
          <AlertCircle className="w-4 h-4 mr-2 text-red-600" />
          {error}
        </div>
      )}

      <div className="bg-white rounded-xl border border-slate-200 shadow-sm p-6 max-w-3xl">
        {loading ? (
          <div className="text-center py-6 text-slate-500 text-sm">Loading configuration rules...</div
>
        ) : (
          <div className="space-y-6">
            {settings.map((s) => (
              <div key={s.setting_key} className="flex flex-col sm:flex-row sm:items-center justify-be
tween pb-4 border-b border-slate-100 last:border-b-0 last:pb-0 gap-3">
                <div>
                  <label className="font-semibold text-sm text-slate-900 block font-mono">
                    {s.setting_key}
                  </label>
                  <span className="text-xs text-slate-500">{s.description}</span>
                </div>
                <div className="flex items-center space-x-2">
                  <input
                    type="text"
                    defaultValue={s.setting_value}
                    id={`input-${s.setting_key}`}
                    disabled={!isAdmin}
                    className="w-32 border border-slate-300 rounded px-3 py-1.5 text-sm text-right fon
t-medium focus:ring-2 focus:ring-red-600 focus:outline-none"
                  />
                  {isAdmin && (
                    <button
                      onClick={() => {
                        const input = document.getElementById(`input-${s.setting_key}`) as HTMLInputEl
ement;
                        if (input) handleUpdate(s.setting_key, input.value);
                      }}
                      className="p-2 bg-slate-900 hover:bg-slate-800 text-white rounded text-xs transi
tion"
                      title="Save Setting"
                    >
                      <Save className="w-4 h-4" />
                    </button>
                  )}
                </div>
              </div>
            ))}
          </div>
        )}
      </div>
    </div>
  );
};
```